

Full Stack Development

Complete Course Notes: Unit I & Unit II

Unit I: Introduction to Web Development & Version Control

Module 1: Introduction to Web Development

Unit I: Introduction to Web Development & Version Control

Definition

Web development is the process of building, creating, and maintaining websites and web applications that run on the internet or an intranet.

It involves both client-side (frontend) technologies that run in the browser and server-side (backend) technologies that run on a web server.

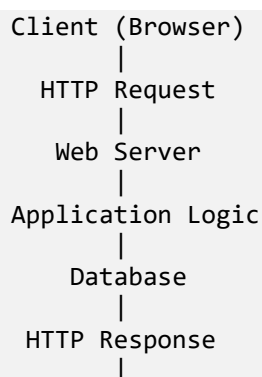
Why is it Needed?

- Enables businesses and individuals to establish an online presence.
- Provides platforms for communication, commerce, and information sharing.
- Supports interactive and dynamic user experiences.
- Forms the foundation for modern software such as web apps, mobile backends, and APIs.
- Is an essential skill for building scalable digital solutions.

Key Points

- Web development consists of Frontend (client-side) and Backend (server-side).
- Frontend: HTML, CSS, JavaScript — runs inside the browser.
- Backend: server, application logic, and database — runs on the server.
- Static Website: fixed content that is the same for every visitor (HTML/CSS only).
- Dynamic Website: content changes based on user or data (requires backend and database).
- The browser sends a request; the web server processes it and sends back a response.

Diagram / Flowchart



Client (Browser)

Syntax

https://www.example.com/products?id=101

Protocol	Domain	Path	Query Parameter

Example Program: Simple Static Web Page

```
<!-- index.html: A simple static webpage -->
<!DOCTYPE html>
<html>
<head>
  <title>My First Website</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is a static web page.</p>
</body>
</html>
```

Output

Welcome to My Website
This is a static web page.

Real-World Example

A college website's "About Us" page is static, showing the same content to every visitor. In contrast, a student portal that shows personalised attendance and marks is dynamic, since it fetches data from a database for each logged-in student.

Advantages

- Enables global reach and round-the-clock accessibility.
- Supports interactive and personalised user experiences.
- Cost-effective way to share information and services.
- Enables e-commerce, online banking, and remote services.
- Continuously evolving with new tools, libraries, and frameworks.

Limitations

- Requires knowledge of multiple technologies (HTML, CSS, JavaScript, backend, database).
- Cross-browser compatibility issues can arise.
- Security and performance require constant attention.

Exam Focus

- Frontend = client-side; Backend = server-side.
- Static website has fixed content; Dynamic website generates content based on data.
- Client-Server architecture is the foundation of the web.
- The HTTP Request/Response cycle is a frequently tested concept.

Examination Questions

1-Mark Questions:

1. What is web development?
2. Define client-server architecture.
3. What is a static website?
4. What is a dynamic website?
5. Expand HTTP.

2-Mark Questions:

1. Differentiate between frontend and backend development.
2. Differentiate between static and dynamic websites.
3. Explain the role of a web server.
4. List the components involved in client-server architecture.

4-Mark Questions:

1. Explain the working of client-server architecture with a diagram.
2. Differentiate between static and dynamic websites with examples.
3. Explain the HTTP request-response cycle with a suitable diagram.

Module 2: HTML Fundamentals (Revision)

Unit I: Introduction to Web Development & Version Control

Definition

HTML (HyperText Markup Language) is the standard markup language used to create the structure and content of web pages using elements called tags.

Why is it Needed?

- Provides the basic structure, or skeleton, of every web page.
- Defines content such as text, images, links, tables, and forms.
- Works together with CSS and JavaScript to build complete web pages.
- Understood by every web browser, making it universally compatible.

Key Points

- HTML documents are made up of elements formed by tags, such as <p> and <h1>.
- Basic document structure: <html>, <head>, <body>.
- Attributes provide extra information about an element (e.g. src, href, alt).
- Headings range from <h1> to <h6>; paragraphs use <p>.
- Lists: ordered , unordered , and list items .
- Tables are built using <table>, <tr>, <td>, and <th>.
- Forms are built using <form>, <input>, <label>, and <button>.
- Images use ; links use .

Diagram / Flowchart

```
<html>
|
+-- <head>
|   +-- <title>
|   +-- <meta>
|
+-- <body>
    +-- <h1> Heading
    +-- <p> Paragraph
    +-- <img>
    +-- <a> Link
```

Syntax

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  <h1>Heading</h1>
  <p>Paragraph text</p>
  <a href="https://example.com">Link</a>
  
</body>
```

```
</html>
```

Example Program: Displaying Student Details

```
<!-- student.html: displays student details using HTML tags -->
<!DOCTYPE html>
<html>
<head>
  <title>Student Details</title>
</head>
<body>
  <h1>Student Management System</h1>
  <p>Name: Ananya Sharma</p>
  <table border="1">
    <tr>
      <th>Subject</th>
      <th>Marks</th>
    </tr>
    <tr>
      <td>Mathematics</td>
      <td>92</td>
    </tr>
  </table>
</body>
</html>
```

Output

```
Student Management System
Name: Ananya Sharma
Subject      Marks
Mathematics  92
```

Real-World Example

A library management system uses HTML tables to list book titles, authors, and availability status on its catalogue page.

Advantages

- Simple and easy to learn.
- Supported by every web browser.
- Forms the base on which CSS and JavaScript are added.
- Lightweight and fast to load.

Limitations

- Cannot perform styling or logic on its own.
- Static in nature; needs CSS, JavaScript, or a backend for dynamic behaviour.

Exam Focus

- HTML stands for HyperText Markup Language.
- Basic structure: html contains head and body.
- The difference between a tag and an attribute is frequently asked.
- Ordered lists use numbers; unordered lists use bullets.

Examination Questions

1-Mark Questions:

6. Expand HTML.
7. What is a tag?
8. What is an attribute?
9. Name the tag used to insert an image.
10. Which tag is used to create a hyperlink?

2-Mark Questions:

5. Differentiate between ordered and unordered lists.
6. List any four commonly used HTML tags with their purpose.
7. Explain the basic structure of an HTML document.

4-Mark Questions:

4. Explain the structure of an HTML document with a diagram and example.
5. Write an HTML program to create a table and explain each tag used.

Module 3: CSS Fundamentals (Revision)

Unit I: Introduction to Web Development & Version Control

Definition

CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation, layout, and visual styling of HTML elements on a web page.

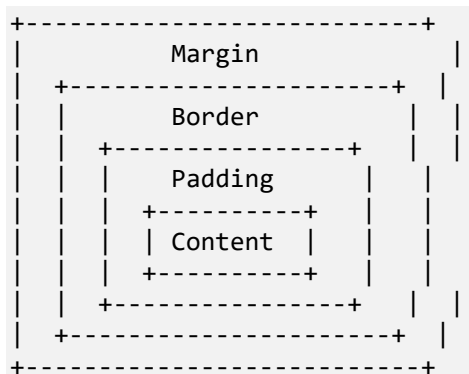
Why is it Needed?

- Separates content (HTML) from presentation (styling).
- Allows consistent styling to be applied across multiple pages.
- Enables control over colors, fonts, spacing, and layout.
- Improves the overall user experience and visual appeal of a website.

Key Points

- Basic CSS syntax: selector { property: value; }
- CSS can be applied in three ways: inline, internal, and external.
- Common selectors: element, class (.name), and id (#name).
- Box Model consists of content, padding, border, and margin.
- The display property controls how an element is rendered (block, inline, flex, grid, none).
- Colors can be defined using names, HEX codes, RGB, or HSL values.

Diagram / Flowchart



Syntax

```
/* External CSS syntax */
selector {
  property: value;
}

/* Examples */
h1 {
  color: blue;
  font-size: 24px;
}

.card {
  background-color: #f2f2f2;
```

```
padding: 10px;
}

#header {
  text-align: center;
}
```

Example Program: Styling a Hospital Website Page

```
<!-- style-demo.html -->
<!DOCTYPE html>
<html>
<head>
  <style>
    /* Internal CSS styling */
    h1 {
      color: darkblue;
      text-align: center;
    }
    p {
      font-family: Arial;
      color: gray;
    }
  </style>
</head>
<body>
  <h1>Welcome to Our Hospital</h1>
  <p>Providing quality healthcare since 1990.</p>
</body>
</html>
```

Output

Welcome to Our Hospital (centered, dark blue heading)
Providing quality healthcare since 1990. (gray Arial text)

Real-World Example

A hospital management system website uses CSS to style patient forms, headers, and appointment tables consistently across all of its pages.

Advantages

- Separates structure from design, improving maintainability.
- Enables reusable styles across many pages through external CSS.
- Provides fine control over layout and appearance.
- Supports responsive and adaptive designs.

Limitations

- Different browsers may render CSS slightly differently.
- Complex layouts can require advanced knowledge of Flexbox or Grid.

Exam Focus

- CSS stands for Cascading Style Sheets.
- Box Model order, from inside out: Content, Padding, Border, Margin.
- Three types of CSS: inline, internal, and external.

- A class selector uses a full stop (.); an ID selector uses a hash (#).

Examination Questions

1-Mark Questions:

11. Expand CSS.
12. What is a CSS selector?
13. Name the three ways to apply CSS to a webpage.
14. What symbol represents a class selector?
15. What symbol represents an ID selector?

2-Mark Questions:

8. Explain the CSS Box Model with its components.
9. Differentiate between class and id selectors.
10. List any three CSS properties used to style text.

4-Mark Questions:

6. Explain the CSS Box Model with a diagram and example.
7. Differentiate between inline, internal, and external CSS with examples.

Module 4: Responsive Web Design

Unit I: Introduction to Web Development & Version Control

Definition

Responsive Web Design is an approach to web design that makes web pages render well on different devices and screen sizes by using flexible layouts, images, and CSS media queries.

Why is it Needed?

- Users access websites from mobiles, tablets, laptops, and desktops.
- Ensures a consistent user experience across all screen sizes.
- Improves accessibility and usability of a website.
- Search engines favour mobile-friendly websites, improving SEO.

Key Points

- Achieved using media queries, flexible grids, and flexible images.
- The viewport meta tag controls scaling on mobile devices.
- Mobile-First Design: design for the smallest screen first, then scale up.
- Breakpoints define screen-width ranges for different layouts.
- Relative units (% , em, rem, vw, vh) are preferred over fixed units (px).

Diagram / Flowchart

```
Desktop (large screen)
  |  media query
  v
Tablet (medium screen)
  |  media query
  v
Mobile (small screen)
```

Syntax

```
<!-- Viewport meta tag -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

/* Media query example */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

Example Program: Responsive Box Example

```
<!-- responsive.html -->
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    .box {
```

```

        background-color: lightblue;
        padding: 20px;
    }
    /* Changes layout on small screens */
    @media (max-width: 600px) {
        .box {
            background-color: lightgreen;
        }
    }
</style>
</head>
<body>
    <div class="box">Resize the window to see the color change</div>
</body>
</html>

```

Output

Box appears light blue on large screens and light green when the browser width is 600px or less.

Real-World Example

An e-commerce website adjusts its product grid from four columns on desktop to a single column on mobile, so users can shop comfortably on any device.

Advantages

- A single website works on all devices, avoiding the need for a separate mobile site.
- Improves user experience and engagement.
- Better SEO ranking on mobile searches.
- Reduces overall development and maintenance cost.

Limitations

- Testing across many devices and screen sizes can be time-consuming.
- Complex responsive designs may increase CSS complexity.

Exam Focus

- Media queries are the core tool used in responsive design.
- Mobile-First approach designs for the smallest screen first.
- The viewport meta tag is essential for correct mobile scaling.
- Relative units are preferred over fixed pixel units.

Examination Questions

1-Mark Questions:

16. What is responsive web design?
17. What is a media query?
18. Define Mobile-First design.
19. What is the purpose of the viewport meta tag?

2-Mark Questions:

11. Explain the role of media queries with an example.

12. Differentiate between Mobile-First and Desktop-First design.

13. List any three relative CSS units used for responsiveness.

4-Mark Questions:

8. Explain responsive web design with a diagram, syntax, and example.

9. Discuss the advantages of responsive web design for real-world applications.

Module 5: CSS Frameworks (Bootstrap)

Unit I: Introduction to Web Development & Version Control

Definition

Bootstrap is a popular open-source CSS framework that provides pre-built CSS and JavaScript components, helping developers quickly build responsive, mobile-first websites.

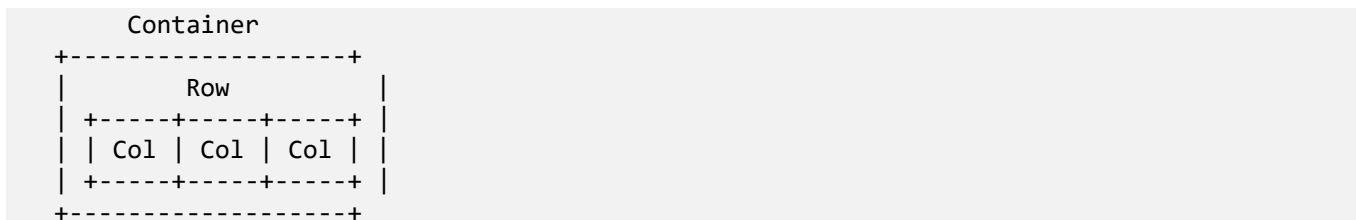
Why is it Needed?

- Speeds up UI development using ready-made components.
- Provides a consistent, responsive grid system.
- Reduces the need to write custom CSS from scratch.
- Ensures cross-browser compatibility.

Key Points

- Bootstrap uses a 12-column responsive Grid System.
- Core structure: container > row > column.
- Provides pre-styled components: buttons, navbars, cards, modals, and forms.
- Includes responsive utility classes such as d-none, d-flex, and text-center.
- Can be added to a project via a CDN link or an npm package.

Diagram / Flowchart



Syntax

```
<!-- Including Bootstrap via CDN -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

<!-- Basic grid syntax -->
<div class="container">
  <div class="row">
    <div class="col">Column 1</div>
    <div class="col">Column 2</div>
  </div>
</div>
```

Example Program: Bootstrap Buttons in a Grid

```
<!-- bootstrap-demo.html -->
<!DOCTYPE html>
<html>
<head>
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container mt-4">
    <div class="row">
      <div class="col-md-6">
        <button class="btn btn-primary">Login</button>
      </div>
      <div class="col-md-6">
        <button class="btn btn-secondary">Register</button>
      </div>
    </div>
  </div>
</body>
</html>
```

Output

Displays two responsive columns, each containing a styled Bootstrap button ("Login" and "Register") side by side.

Real-World Example

A banking application's login page uses Bootstrap's grid and form components to quickly build a clean, responsive layout without writing custom CSS.

Advantages

- Speeds up development with ready-made components.
- Ensures responsive design out of the box.
- Large community support and documentation.
- Consistent look and feel across the application.
- Easy to customize using utility classes.

Limitations

- Websites can look generic if not customized.
- Adds extra CSS/JS file size if unused components are included.

Exam Focus

- The Bootstrap Grid System is based on 12 columns.
- Structure: container > row > column.
- Bootstrap can be included via a CDN or installed via npm.
- Utility classes help avoid writing custom CSS.

Examination Questions

1-Mark Questions:

20. What is Bootstrap?
21. How many columns does the Bootstrap grid system use?
22. Name any two Bootstrap components.
23. How can Bootstrap be added to a project?

2-Mark Questions:

14. Explain the container-row-column structure in Bootstrap.

15. List any four advantages of using Bootstrap.

16. What are Bootstrap utility classes?

4-Mark Questions:

10. Explain the Bootstrap Grid System with a diagram and example.

11. Discuss the advantages and limitations of using Bootstrap in web development.

Module 6: Advanced CSS (Flexbox & CSS Grid)

Unit I: Introduction to Web Development & Version Control

Definition

Flexbox and CSS Grid are modern CSS layout modules used to arrange and align elements efficiently: Flexbox in one dimension, and Grid in two dimensions.

Why is it Needed?

- Simplifies complex layout design compared to older methods such as floats and tables.
- Provides easy alignment and spacing of elements.
- Essential for building responsive and adaptive layouts.
- Reduces dependency on external CSS frameworks.

Key Points

- Flexbox arranges items in a single dimension: a row or a column.
- display: flex activates Flexbox on a container.
- Common Flexbox properties: justify-content, align-items, flex-direction.
- CSS Grid arranges items in two dimensions: rows and columns together.
- display: grid activates Grid on a container.
- Common Grid properties: grid-template-columns, grid-template-rows, gap.

Diagram / Flowchart

```
Flexbox (1D - row)
[ Item1 ][ Item2 ][ Item3 ]
```

```
CSS Grid (2D - rows & columns)
[ Item1 ][ Item2 ]
[ Item3 ][ Item4 ]
```

Syntax

```
/* Flexbox */
.container {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* CSS Grid */
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

Example Program: Photo Gallery Using CSS Grid

```
<!-- grid-demo.html -->
<!DOCTYPE html>
<html>
```

```
<head>
  <style>
    .gallery {
      display: grid;
      grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
      gap: 10px;
    }
    .gallery div {
      background-color: lightcoral;
      padding: 20px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="gallery">
    <div>Image 1</div>
    <div>Image 2</div>
    <div>Image 3</div>
  </div>
</body>
</html>
```

Output

Displays three equally sized boxes ("Image 1", "Image 2", "Image 3") arranged neatly in a row using CSS Grid.

Real-World Example

A social media application uses Flexbox to align profile icons and buttons in a navbar, and CSS Grid to arrange photo galleries in a neat grid layout.

Advantages

- Simplifies alignment and spacing without extra markup.
- Reduces the need for floats and positioning hacks.
- Grid allows precise two-dimensional layout control.
- Both are widely supported in modern browsers.
- Makes responsive design significantly easier to implement.

Limitations

- Requires a different mental model than traditional CSS and takes time to master.
- Not supported in very old browsers, though this is rarely an issue today.

Exam Focus

- Flexbox is one-dimensional; Grid is two-dimensional.
- `display: flex` and `display: grid` activate each layout mode.
- `justify-content` aligns items horizontally in Flexbox.
- `grid-template-columns` defines the column structure in Grid.

Examination Questions

1-Mark Questions:

- 24. What is Flexbox used for?
- 25. What is CSS Grid used for?
- 26. Which property activates Flexbox on a container?
- 27. Which property activates CSS Grid on a container?

2-Mark Questions:

- 17. Differentiate between Flexbox and CSS Grid.
- 18. Explain the purpose of justify-content and align-items.
- 19. List any three properties used in CSS Grid.

4-Mark Questions:

- 12. Explain Flexbox with syntax, diagram, and a suitable example.
- 13. Explain CSS Grid with syntax, diagram, and a suitable example.
- 14. Compare Flexbox and CSS Grid with real-world use cases.

Module 7: JavaScript Basics

Unit I: Introduction to Web Development & Version Control

Definition

JavaScript is a lightweight, interpreted programming language that adds interactivity and dynamic behaviour to web pages, running directly inside the browser.

Why is it Needed?

- Makes static web pages interactive and dynamic.
- Allows manipulation of HTML and CSS at runtime.
- Enables client-side validation and business logic.
- Forms the foundation for modern frameworks such as React.
- Runs directly in the browser without needing compilation.

Key Points

- JavaScript is primarily a client-side scripting language, but can also run server-side using Node.js.
- Variables are declared using var, let, or const.
- let and const are block-scoped (ES6+); var is function-scoped.
- const cannot be reassigned after declaration; let can be reassigned.
- Data types: Number, String, Boolean, Undefined, Null, Object, Symbol.
- Operators: Arithmetic, Comparison, Logical, and Assignment operators.

Diagram / Flowchart

```
<script> Tag / .js File
|
JS Engine (Browser)
|
Parsing & Execution
|
Updates DOM / Output
```

Syntax

```
// Variable declaration
let age = 20;
const name = "Riya";
var city = "Delhi"; // older way

// Data types
let isStudent = true;      // Boolean
let marks = 95;            // Number
let course = "BCA";       // String
```

Example Program: Displaying Student Marks

```
// basics.js
let studentName = "Karan";
let marks = 78;

// Template literal to combine text and variables
```

```
console.log(`${studentName} scored ${marks} marks`);
```

Output

```
Karan scored 78 marks
```

Real-World Example

A banking application uses JavaScript to instantly validate whether an entered account number has the correct number of digits before the form is submitted.

Advantages

- Runs directly in the browser; no compilation needed.
- Makes web pages interactive and dynamic.
- Supported by all modern browsers.
- Huge ecosystem of libraries and frameworks.
- Can be used for both frontend and backend development (via Node.js).

Limitations

- Different browsers may occasionally execute JavaScript slightly differently.
- Its single-threaded nature can make heavy computations block the user interface.

Exam Focus

- var is function-scoped; let and const are block-scoped.
- const cannot be reassigned after declaration.
- JavaScript is capable of both client-side and server-side execution.
- The distinction between primitive and reference data types is frequently asked.

Examination Questions

1-Mark Questions:

28. What is JavaScript?
29. Name the three ways to declare a variable in JavaScript.
30. Which keyword is used to declare a constant?
31. List any two JavaScript data types.

2-Mark Questions:

20. Differentiate between var, let, and const.
21. List any four data types in JavaScript.
22. What is a template literal? Give an example.

4-Mark Questions:

15. Explain JavaScript variables and data types with examples.
16. Discuss the differences between var, let, and const with a suitable example.

Module 8: Control Flow Statements

Unit I: Introduction to Web Development & Version Control

Definition

Control flow statements determine the order in which statements are executed in a program, using conditions and loops to control the flow of execution.

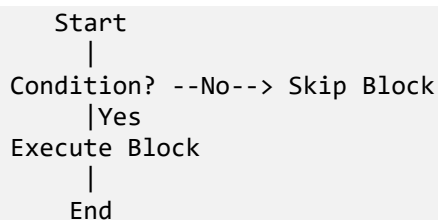
Why is it Needed?

- Enables decision-making in programs using constructs such as if-else and switch.
- Allows repetition of code blocks using loops.
- Essential for implementing logic based on varying conditions.
- Reduces code duplication through iteration.

Key Points

- if, if-else, and else-if are used for conditional branching.
- switch statement handles multiple discrete conditions cleanly.
- for loop is used when the number of iterations is known in advance.
- while loop checks its condition before each iteration.
- do-while loop executes at least once before checking its condition.
- break exits a loop or switch; continue skips to the next iteration.

Diagram / Flowchart



Syntax

```
// if-else
if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}

// switch
switch (day) {
  case 1:
    console.log("Monday");
    break;
  default:
    console.log("Invalid day");
}

// for loop
for (let i = 1; i <= 5; i++) {
```

```
console.log(i);  
}
```

Example Program: Grade Calculator

```
// grade.js  
let marks = 82;  
  
// Determine grade using an if-else ladder  
if (marks >= 90) {  
    console.log("Grade: A");  
} else if (marks >= 75) {  
    console.log("Grade: B");  
} else {  
    console.log("Grade: C");  
}
```

Output

Grade: B

Real-World Example

An online food delivery app uses if-else logic to check whether an order amount qualifies for free delivery (for example, above a set threshold) before displaying the delivery charge.

Advantages

- Allows dynamic decision-making in programs.
- Reduces repetitive code using loops.
- Improves program logic and readability.
- Serves as an essential building block for algorithms.

Exam Focus

- The key difference between while and do-while is that do-while executes at least once.
- switch uses break to prevent fall-through to the next case.
- A for loop combines initialization, condition, and increment/decrement in one line.
- The difference between break and continue is a commonly asked comparison.

Examination Questions

1-Mark Questions:

32. What is a control flow statement?
33. Name any two looping statements in JavaScript.
34. What does the break statement do?
35. What does the continue statement do?

2-Mark Questions:

23. Differentiate between while and do-while loops.
24. Explain the syntax of a for loop.
25. What is the purpose of the switch statement?

4-Mark Questions:

17. Explain if, if-else, and switch statements with examples.

18. Explain for, while, and do-while loops with syntax and examples.

Module 9: Functions

Unit I: Introduction to Web Development & Version Control

Definition

A function is a reusable block of code designed to perform a specific task. It can be called whenever needed, optionally accepting inputs (parameters) and returning a value.

Why is it Needed?

- Promotes code reusability and reduces duplication.
- Breaks a program into smaller, manageable pieces.
- Makes debugging and testing easier.
- Improves overall code organization and readability.

Key Points

- Function Declaration: defined using the function keyword.
- Function Expression: a function assigned to a variable.
- Arrow Function: shorter ES6+ syntax written using `=>`.
- Parameters are the inputs defined in a function; arguments are the actual values passed when calling it.
- The return statement sends a value back to the caller.
- A function can be called multiple times with different arguments.

Diagram / Flowchart

```
graph TD
    Caller --> function_args[function(args)]
    function_args --> Body[Function Body Executes]
    Body --> return_value[return value]
    return_value --> Caller_result[Caller receives result]
```

Syntax

```
// Function Declaration
function greet(name) {
  return `Hello, ${name}`;
}

// Function Expression
const add = function (a, b) {
  return a + b;
};

// Arrow Function
const multiply = (a, b) => a * b;
```

Example Program: Rectangle Area Calculator

```
// calculator.js
```

```
// Arrow function to calculate area of a rectangle
const calculateArea = (length, width) => {
  return length * width;
};

console.log(`Area: ${calculateArea(5, 3)}`);
```

Output

Area: 15

Real-World Example

A banking application uses a function such as `calculateInterest(principal, rate, time)` that can be called repeatedly with different customer values instead of rewriting the formula each time.

Advantages

- Improves code reusability and modularity.
- Makes large programs easier to manage.
- Simplifies debugging and testing.
- Arrow functions offer concise syntax for short operations.

Limitations

- Excessive small functions can make code harder to trace if poorly named.
- Arrow functions do not have their own 'this' binding, which can confuse beginners.

Exam Focus

- The difference between function declaration and function expression is frequently tested.
- Arrow functions use `=>` syntax and have no own 'this'.
- Parameters versus arguments is a common exam distinction.
- The return statement ends function execution and sends back a value.

Examination Questions

1-Mark Questions:

36. What is a function?
37. What is the difference between a parameter and an argument?
38. What symbol is used to define an arrow function?
39. What does the return statement do?

2-Mark Questions:

26. Differentiate between function declaration and function expression.
27. Explain arrow functions with an example.
28. Why are functions important in programming?

4-Mark Questions:

19. Explain functions in JavaScript with syntax and example programs.
20. Discuss function declaration, function expression, and arrow functions with examples.

Module 10: Objects & Arrays

Unit I: Introduction to Web Development & Version Control

Definition

Objects are collections of key-value pairs used to store related data and behaviour, while Arrays are ordered collections of values used to store multiple items under a single variable name.

Why is it Needed?

- Objects allow grouping of related data, such as a student's name, age, and marks, into a single unit.
- Arrays allow efficient storage and management of multiple values, such as a list of marks.
- Both are essential for organizing and processing real-world data.
- Provide built-in methods to search, filter, and transform data easily.

Key Points

- Object syntax uses key: value pairs inside curly braces {}.
- Object properties can be accessed using dot notation or bracket notation.
- Arrays are created using square brackets [].
- Array indexing starts from 0.
- Common array methods include push(), pop(), map(), filter(), and forEach().
- Objects can contain arrays, and arrays can contain objects.

Diagram / Flowchart

```
Object: student
{
  name: "Riya",
  age: 20,
  marks: [88, 92, 79]  <-- Array inside Object
}
```

Syntax

```
// Object
const student = {
  name: "Riya",
  age: 20
};
console.log(student.name); // dot notation

// Array
const marks = [88, 92, 79];
console.log(marks[0]); // bracket indexing

// Array method
const passed = marks.filter(m => m >= 80);
```

Example Program: Calculating a Student's Total Marks

```
// studentData.js
const student = {
  name: "Vikram",
```

```
marks: [75, 82, 90]
};

// Calculate total marks using an array method
const total = student.marks.reduce((sum, m) => sum + m, 0);

console.log(`${student.name}'s total marks: ${total}`);
```

Output

Vikram's total marks: 247

Real-World Example

A student management system stores each student as an object with properties like name and roll number, along with an array of subject-wise marks, making it easy to calculate totals and averages.

Advantages

- Efficiently organizes and stores related data.
- Arrays provide many built-in methods for data processing.
- Objects model real-world entities naturally.
- Easy to loop through and transform data using map, filter, and forEach.

Limitations

- Deeply nested objects and arrays can become difficult to read and debug.
- Improper handling of array indices can cause errors, such as out-of-bounds access.

Exam Focus

- Objects use key-value pairs; arrays use indexed values starting at 0.
- Common array methods: push, pop, map, filter, forEach, reduce.
- Dot notation versus bracket notation for accessing object properties is often asked.
- Arrays and objects can be nested within each other.

Examination Questions

1-Mark Questions:

40. What is an object in JavaScript?
41. What is an array?
42. From which index does array numbering start?
43. Name any two array methods.

2-Mark Questions:

29. Differentiate between an object and an array.
30. Explain dot notation and bracket notation with examples.
31. List any four commonly used array methods with their purpose.

4-Mark Questions:

21. Explain objects and arrays in JavaScript with syntax and examples.
22. Explain any four array methods with suitable example code.

Module 11: Variables & Scope

Unit I: Introduction to Web Development & Version Control

Definition

Scope determines the accessibility, or visibility, of variables within different parts of a program, defining where a variable can be accessed or modified.

Why is it Needed?

- Prevents naming conflicts between variables in different parts of code.
- Controls memory usage by limiting variable lifetime.
- Helps in writing predictable, bug-free code.
- Essential for understanding how JavaScript functions and blocks behave.

Key Points

- Global Scope: a variable accessible throughout the entire program.
- Local/Function Scope: a variable accessible only within the function it is declared in.
- Block Scope: `let` and `const` are limited to the `{ }` block in which they are declared.
- Hoisting: variable and function declarations are moved to the top of their scope during compilation.
- `var` is hoisted and initialized as `undefined`; `let/const` are hoisted but not initialized (Temporal Dead Zone).
- A variable's lifetime ends when execution of its scope completes.

Diagram / Flowchart

```
Global Scope
|
+-- Function Scope
|   |
|   +-- Block Scope { }
|
(variables in inner scope can access outer,
but not vice versa)
```

Syntax

```
let globalVar = "I am global";

function demo() {
  let localVar = "I am local";
  if (true) {
    let blockVar = "I am block-scoped";
    console.log(blockVar); // accessible here
  }
  // console.log(blockVar); // Error: not accessible here
}
```

Example Program: Global vs Local Scope

```
// scopeDemo.js
let count = 10; // global scope

function updateCount() {
```

```
let count = 20; // local scope, different from global
console.log("Inside function:", count);
}

updateCount();
console.log("Outside function:", count);
```

Output

```
Inside function: 20
Outside function: 10
```

Real-World Example

In a hospital management system, a globally scoped variable might store the hospital's name, used everywhere in the application, while a locally scoped variable inside a function might store a temporary value like a single patient's token number.

Advantages

- Prevents accidental overwriting of variables across the program.
- Improves memory management by limiting variable lifetime.
- Makes debugging easier by isolating variables to specific blocks or functions.
- Supports better code organization and modularity.

Limitations

- The hoisting behaviour of var can lead to confusing bugs if not understood.
- Overuse of global variables can make large programs harder to maintain.

Exam Focus

- var is function-scoped; let and const are block-scoped.
- Hoisting moves declarations to the top of their scope.
- The Temporal Dead Zone applies to let and const before initialization.
- Global vs Local vs Block scope is a frequently asked comparison.

Examination Questions

1-Mark Questions:

44. What is scope in JavaScript?
45. Define global scope.
46. Define hoisting.
47. Which variables are block-scoped?

2-Mark Questions:

32. Differentiate between global scope and local scope.
33. Explain hoisting with an example.
34. What is block scope? Give an example.

4-Mark Questions:

23. Explain the different types of scope in JavaScript with examples.
24. Explain hoisting and the Temporal Dead Zone with suitable examples.

Module 12: Error Handling

Unit I: Introduction to Web Development & Version Control

Definition

Error handling is the process of anticipating, detecting, and responding to errors during program execution, using constructs like try...catch to prevent programs from crashing.

Why is it Needed?

- Prevents the entire application from crashing due to unexpected errors.
- Allows graceful handling of invalid input or failed operations.
- Helps developers identify and fix bugs during development.
- Improves the reliability and user experience of applications.

Key Points

- Common errors include `SyntaxError`, `ReferenceError`, and `TypeError`.
- The `try` block contains code that might throw an error.
- The `catch` block handles the error if one occurs.
- The `finally` block executes regardless of whether an error occurred.
- The `throw` statement can be used to create custom errors.
- Browser Developer Tools and the Console help in debugging errors.

Diagram / Flowchart

```
try {  
    Risky Code  
}  
|  
Error Occurred? --No--> Continue Normally  
|Yes  
catch (error)  
|  
Handle Error  
|  
finally { Always Runs }
```

Syntax

```
try {  
    // Code that might throw an error  
    let result = riskyOperation();  
} catch (error) {  
    // Handle the error  
    console.log("Error occurred:", error.message);  
} finally {  
    console.log("Execution completed");  
}
```

Example Program: Safe Division Function

```
// errorDemo.js  
function divide(a, b) {
```

```
try {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
} catch (error) {
  console.log("Error:", error.message);
} finally {
  console.log("Division attempt finished");
}
}

console.log(divide(10, 0));
```

Output

```
Error: Cannot divide by zero
Division attempt finished
undefined
```

Real-World Example

An online banking application uses try...catch to handle situations where a fund transfer fails due to network issues, showing the user a friendly error message instead of crashing the app.

Advantages

- Prevents application crashes due to unexpected errors.
- Improves user experience with meaningful error messages.
- Makes debugging easier during development.
- Allows custom error handling logic using throw.

Limitations

- Overusing try...catch can hide genuine bugs if errors are silently ignored.
- Cannot prevent syntax errors, which occur before code execution.

Exam Focus

- The try...catch...finally structure is important to memorize.
- The finally block always executes, regardless of whether an error occurred.
- throw is used to create custom or user-defined errors.
- Common error types: SyntaxError, ReferenceError, TypeError.

Examination Questions

1-Mark Questions:

48. What is error handling?
49. Name the three blocks used in JavaScript error handling.
50. What does the throw statement do?
51. Name any two common JavaScript error types.

2-Mark Questions:

35. Explain the purpose of try, catch, and finally blocks.
36. Differentiate between syntax errors and runtime errors.

37. What debugging tools are commonly used in browsers?

4-Mark Questions:

25. Explain error handling in JavaScript using try...catch...finally with an example.

26. Discuss common JavaScript errors and debugging techniques used to resolve them.

Module 13: DOM Manipulation

Unit I: Introduction to Web Development & Version Control

Definition

The DOM (Document Object Model) is a tree-like representation of an HTML document that JavaScript can use to access, modify, add, or remove elements and content dynamically.

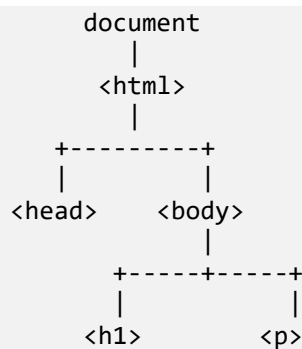
Why is it Needed?

- Allows JavaScript to interact with and modify web page content after it loads.
- Enables dynamic updates without reloading the page.
- Essential for building interactive user interfaces.
- Forms the basis for how frameworks like React update the UI.

Key Points

- The document object is the entry point for accessing the DOM.
- Selecting elements: `getElementById`, `querySelector`, `querySelectorAll`, `getElementsByClassName`.
- Changing content: `innerHTML` and `textContent`.
- Modifying attributes: `setAttribute()` and `getAttribute()`.
- Creating elements: `createElement()` and `appendChild()`.
- Removing elements: `removeChild()` and `remove()`.

Diagram / Flowchart



Syntax

```
// Selecting elements
const title = document.getElementById("title");
const items = document.querySelectorAll(".item");

// Changing content
title.textContent = "New Heading";

// Modifying attributes
title.setAttribute("class", "highlight");

// Creating and appending an element
const newPara = document.createElement("p");
newPara.textContent = "New paragraph";
document.body.appendChild(newPara);
```

Example Program: Updating a Welcome Message

```
// domDemo.js
// Selects the element with id "message" and updates its text
const message = document.getElementById("message");
message.textContent = "Welcome to the Student Portal!";

// Change its background color
message.style.backgroundColor = "lightyellow";
```

Output

The element with id "message" now displays "Welcome to the Student Portal!" with a light yellow background, without reloading the page.

Real-World Example

A student portal dynamically updates the "Notifications" section on the dashboard using DOM manipulation whenever a new notice is added, without refreshing the entire page.

Advantages

- Enables dynamic, real-time content updates.
- Improves user experience by avoiding full page reloads.
- Provides fine-grained control over HTML elements.
- Works natively in all modern browsers.

Limitations

- Frequent, unoptimized DOM manipulation can slow down performance.
- Directly manipulating the DOM in large applications can become hard to manage (this is one reason frameworks like React exist).

Exam Focus

- DOM stands for Document Object Model.
- `getElementById` versus `querySelector` is a common comparison.
- The difference between `innerHTML` and `textContent` is frequently asked.
- The DOM is represented as a tree structure.

Examination Questions

1-Mark Questions:

52. Expand DOM.
53. Name any two methods used to select elements in the DOM.
54. What is the use of `innerHTML`?
55. Which method is used to create a new element?

2-Mark Questions:

38. Differentiate between `getElementById` and `querySelector`.
39. Differentiate between `innerHTML` and `textContent`.
40. Explain how to add a new element to the DOM.

4-Mark Questions:

27. Explain the DOM with a diagram and describe common methods used to manipulate it.
28. Write a JavaScript program to select, modify, and create DOM elements with explanation.

Module 14: Event Handling

Unit I: Introduction to Web Development & Version Control

Definition

Event handling refers to writing code that responds to user actions or browser events, such as clicks, key presses, or form submissions, by executing specific functions called event handlers.

Why is it Needed?

- Enables interactivity by responding to user actions.
- Essential for building forms, buttons, and interactive UI elements.
- Allows validation and dynamic behaviour based on user input.
- Forms the foundation for event-driven programming used in React and other frameworks.

Key Points

- Mouse Events: click, dblclick, mouseover, mouseout.
- Keyboard Events: keydown, keyup, keypress.
- Form Events: submit, change, input, focus, blur.
- `addEventListener()` is the standard method to attach an event handler.
- The event object provides details about the event, such as `event.target`.
- `preventDefault()` stops an event's default browser behaviour, such as form submission.

Diagram / Flowchart

```
graph TD
    A[User Action (click, keypress, etc.)] --> B[Event Triggered]
    B --> C[Event Listener Detects It]
    C --> D[Event Handler Function Runs]
```

Syntax

```
// Attaching an event listener
button.addEventListener("click", function () {
  console.log("Button clicked!");
});

// Using an arrow function
input.addEventListener("keyup", (event) => {
  console.log(event.target.value);
});
```

Example Program: Handling a Login Button Click

```
// eventDemo.js
const loginButton = document.getElementById("loginBtn");

// Runs when the button is clicked
loginButton.addEventListener("click", () => {
  alert("Login button clicked!");
});
```

```
});
```

Output

Clicking the button with id "loginBtn" displays an alert box saying "Login button clicked!".

Real-World Example

An e-commerce website uses event handling to detect when a user clicks the "Add to Cart" button and instantly updates the cart count without reloading the page.

Advantages

- Makes web pages interactive and responsive to users.
- Supports a wide variety of event types (mouse, keyboard, form).
- Enables real-time validation and feedback.
- Essential for building modern, dynamic web applications.

Limitations

- Too many event listeners on a page can affect performance if not managed properly.
- Improper removal of event listeners can cause memory leaks.

Exam Focus

- `addEventListener()` is the standard way to handle events.
- `preventDefault()` stops default browser actions.
- Types of events: mouse, keyboard, and form events.
- `event.target` refers to the element that triggered the event.

Examination Questions

1-Mark Questions:

56. What is event handling?
57. Name any two mouse events.
58. Which method is used to attach an event listener?
59. What does `preventDefault()` do?

2-Mark Questions:

41. Explain the syntax of `addEventListener()` with an example.
42. List any four types of events in JavaScript.
43. What is the purpose of the event object?

4-Mark Questions:

29. Explain event handling in JavaScript with syntax and a suitable example.
30. Discuss different types of events with examples for each.

Module 15: Debugging JavaScript

Unit I: Introduction to Web Development & Version Control

Definition

Debugging is the process of identifying, analyzing, and fixing errors or unexpected behaviour in JavaScript code, often using browser developer tools.

Why is it Needed?

- Helps identify and fix bugs quickly during development.
- Improves code quality and reliability.
- Saves development time by pinpointing exact error locations.
- Essential skill for building maintainable applications.

Key Points

- Browser Developer Tools (DevTools) can be opened using F12 or right-click > Inspect.
- The Console tab displays errors, warnings, and console.log() output.
- Breakpoints allow pausing code execution at a specific line to inspect values.
- Step Over, Step Into, and Step Out are used to navigate through code during debugging.
- console.log(), console.error(), and console.table() help track variable values.
- The Network tab helps debug API calls and requests.

Diagram / Flowchart

```
graph TD
    A[Open DevTools (F12)] --> B[Go to Console / Sources Tab]
    B --> C[Set Breakpoint on a Line]
    C --> D[Run/Reload the Page]
    D --> E[Execution Pauses at Breakpoint]
    E --> F[Inspect Variables & Fix Bug]
```

Syntax

```
// Basic debugging using console
console.log("Value of x:", x);
console.error("Something went wrong");
console.table([ { name: "Riya", marks: 90 } ]);

// Manually pausing execution
debugger;
```

Example Program: Debugging a Total Calculation

```
// debugDemo.js
function calculateTotal(price, qty) {
  console.log("price:", price, "qty:", qty); // debug log
  return price * qty;
}
```

```
}  
  
console.log(calculateTotal(100, 3));
```

Output

```
price: 100 qty: 3  
300
```

Real-World Example

A developer debugging a banking application's interest calculator uses breakpoints in DevTools to inspect why the calculated interest is showing as "NaN" (Not a Number).

Advantages

- Helps quickly locate and fix errors in code.
- Reduces time spent manually tracing bugs.
- Provides detailed insight into variable values and execution flow.
- Built into every modern browser at no extra cost.

Exam Focus

- DevTools can be opened using F12 or Inspect.
- Breakpoints pause code execution for inspection.
- `console.log()` is the most common debugging tool.
- The debugger keyword pauses execution when DevTools is open.

Examination Questions

1-Mark Questions:

60. What is debugging?
61. How do you open browser Developer Tools?
62. What is a breakpoint?
63. Name any one console method used for debugging.

2-Mark Questions:

44. Explain the purpose of breakpoints in debugging.
45. List any four features of browser Developer Tools.
46. What is the use of the debugger keyword?

4-Mark Questions:

31. Explain the process of debugging JavaScript code using browser Developer Tools.
32. Discuss common debugging techniques with suitable examples.

Module 16: Introduction to jQuery

Unit I: Introduction to Web Development & Version Control

Definition

jQuery is a fast, lightweight JavaScript library that simplifies HTML DOM manipulation, event handling, and animation using a concise, easy-to-use syntax.

Why is it Needed?

- Simplifies complex JavaScript/DOM operations into fewer lines of code.
- Handles cross-browser inconsistencies automatically.
- Speeds up development of interactive web pages.
- Widely used in legacy and existing web projects.

Key Points

- jQuery syntax follows the pattern `$(selector).action()`.
- Selectors work similarly to CSS selectors, e.g. `$("#id")`, `$(".class")`.
- Common methods: `.html()`, `.text()`, `.css()`, `.hide()`, `.show()`.
- Event handling is simplified using `.click()` and `.on()`.
- The jQuery library must be included via CDN or download before use.

Diagram / Flowchart

```
$(selector)
  |
.action()
  |
DOM Element Updated
```

Syntax

```
<!-- Including jQuery via CDN -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

// jQuery selector and method
$("#title").text("Updated Title");
$(".box").css("color", "blue");
$("#btn").click(function () {
    alert("Button clicked using jQuery!");
});
```

Example Program: Changing Text on Button Click

```
<!-- jqueryDemo.html -->
<!DOCTYPE html>
<html>
<head>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
    <p id="greeting">Hello</p>
    <button id="changeBtn">Change Text</button>
```

```
<script>
  // When button is clicked, update the paragraph text
  $("#changeBtn").click(function () {
    $("#greeting").text("Hello, jQuery!");
  });
</script>
</body>
</html>
```

Output

Clicking "Change Text" updates the paragraph text from "Hello" to "Hello, jQuery!".

Real-World Example

Many older library management system websites use jQuery to simplify tasks such as showing or hiding search filters, or animating dropdown menus.

Advantages

- Simplifies DOM manipulation with shorter syntax.
- Handles cross-browser compatibility automatically.
- Large number of plugins available for animations and UI components.
- Easy to learn for those already familiar with CSS selectors.

Limitations

- Modern JavaScript (ES6+) and frameworks like React have reduced the need for jQuery.
- Adds extra library overhead and file size for simple tasks.
- Not recommended for large-scale modern applications built with component-based frameworks.

Exam Focus

- jQuery syntax: \$(selector).action().
- The jQuery library must be included before it can be used.
- Common methods: .html(), .text(), .css(), .click().
- jQuery simplifies DOM manipulation compared to plain JavaScript.

Examination Questions

1-Mark Questions:

64. What is jQuery?
65. What is the basic syntax structure of jQuery?
66. Name any two jQuery methods.
67. How is jQuery included in a webpage?

2-Mark Questions:

47. Explain jQuery selectors with an example.
48. List any four advantages of using jQuery.
49. Differentiate between plain JavaScript and jQuery for DOM manipulation.

4-Mark Questions:

33. Explain jQuery with syntax, diagram, and a suitable example.

34. Discuss the advantages and limitations of using jQuery in modern web development.

Module 17: Git & Version Control

Unit I: Introduction to Web Development & Version Control

Definition

Version control is a system that records changes to files over time, allowing developers to track history and collaborate. Git is the most widely used distributed version control system.

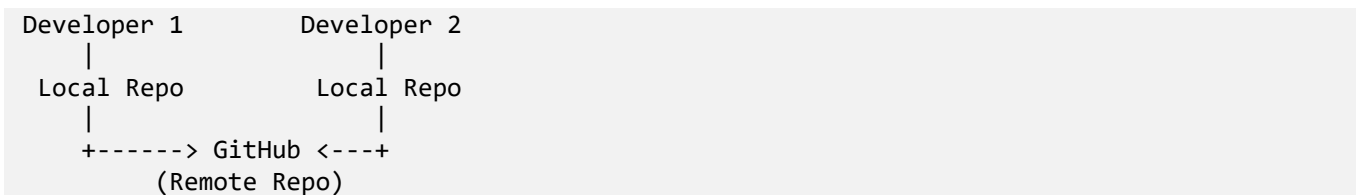
Why is it Needed?

- Tracks changes made to code over time.
- Allows multiple developers to collaborate without overwriting each other's work.
- Enables reverting to previous versions if something goes wrong.
- Essential for teamwork in real-world software projects.

Key Points

- Git is a Distributed Version Control System (DVCS); every developer has a full copy of the repository.
- GitHub is a cloud platform used to host Git repositories online.
- Git is the tool, while GitHub is a hosting service built around Git.
- Key benefits: history tracking, collaboration, branching, and backup.
- A repository (repo) is a project folder tracked by Git.

Diagram / Flowchart



Syntax

```
# Check installed Git version
git --version
```

Example Program: Basic Git Workflow Overview

```
# 1. Initialize a repository
git init

# 2. Track changes and save them
git add .
git commit -m "Initial commit"

# 3. Share changes on GitHub
git push origin main
```

Output

```
Initialized empty Git repository
[main (root-commit) a1b2c3d] Initial commit
```

Real-World Example

In a hospital management system project built by a team of five developers, Git allows each developer to work on different modules, such as billing and patient records, simultaneously without overwriting each other's code, while GitHub hosts the shared repository.

Advantages

- Tracks the complete history of code changes.
- Enables safe collaboration among multiple developers.
- Allows easy rollback to previous versions.
- Supports branching for parallel feature development.
- Free and open-source, with huge community support.

Exam Focus

- Git is the version control tool; GitHub is a hosting platform for Git repositories.
- Git is a Distributed Version Control System (DVCS).
- The difference between Git and GitHub is a very frequently asked question.
- A repository is a project folder tracked by Git.

Examination Questions

1-Mark Questions:

68. What is version control?
69. What is Git?
70. What is GitHub?
71. Define a repository.

2-Mark Questions:

50. Differentiate between Git and GitHub.
51. List any four benefits of using Git.
52. What is meant by Distributed Version Control System?

4-Mark Questions:

35. Explain version control and the need for Git with a diagram.
36. Discuss the benefits of Git in team-based software development.

Module 18: Git Repository

Unit I: Introduction to Web Development & Version Control

Definition

A Git repository is a storage location where a project's files, along with their complete history of changes, are tracked and managed using Git.

Why is it Needed?

- Provides a structured way to store and track project files.
- Maintains a complete history of every change made.
- Allows configuration of user identity for tracking contributions.
- Forms the base requirement before using any Git commands.

Key Points

- Installing Git: downloaded from git-scm.com and installed on the system.
- Creating a repository: git init initializes a new local repository.
- Cloning a repository: git clone <url> copies an existing remote repository.
- Git Configuration sets the username and email used in commits.
- The .git folder is a hidden folder created inside a repository that stores all version history.

Diagram / Flowchart

```
graph TD
    A[Project Folder] --> B[git init]
    B --> C[.git Folder Created]
    C --> D[Repository Ready to Track Changes]
```

Syntax

```
# Configure user identity (one-time setup)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Create a new repository
git init

# Clone an existing repository
git clone https://github.com/username/repo.git
```

Example Program: Creating and Configuring a New Repository

```
# Creating and configuring a new repository
mkdir student-portal
cd student-portal
git init
git config user.name "Ananya"
git config user.email "ananya@example.com"
```

Output

```
Initialized empty Git repository in student-portal/.git/
```

Real-World Example

A team starting a new online food delivery project first creates a Git repository using `git init` (or clones an existing one from GitHub) so that all developers can begin tracking their contributions from day one.

Advantages

- Provides a reliable, centralized way to track project history.
- Easy to set up with a single command (`git init`).
- Configuration ensures each contributor's changes are properly credited.
- Works seamlessly with hosting platforms like GitHub.

Exam Focus

- `git init` creates a new local repository.
- `git clone` copies an existing remote repository.
- `git config` sets the username and email used for commits.
- The `.git` folder stores all version history and must not be deleted.

Examination Questions

1-Mark Questions:

72. What is a Git repository?
73. Which command initializes a new Git repository?
74. Which command is used to copy an existing repository?
75. What is the purpose of `git config`?

2-Mark Questions:

53. Explain the steps to create and configure a new Git repository.
54. Differentiate between `git init` and `git clone`.
55. What is the purpose of the `.git` folder?

4-Mark Questions:

37. Explain the steps to install Git and create a repository with syntax and example.
38. Discuss the importance of Git configuration in collaborative projects.

Module 19: Git Commands

Unit I: Introduction to Web Development & Version Control

Definition

Git commands are command-line instructions used to perform version control operations such as tracking changes, saving snapshots, and viewing project history.

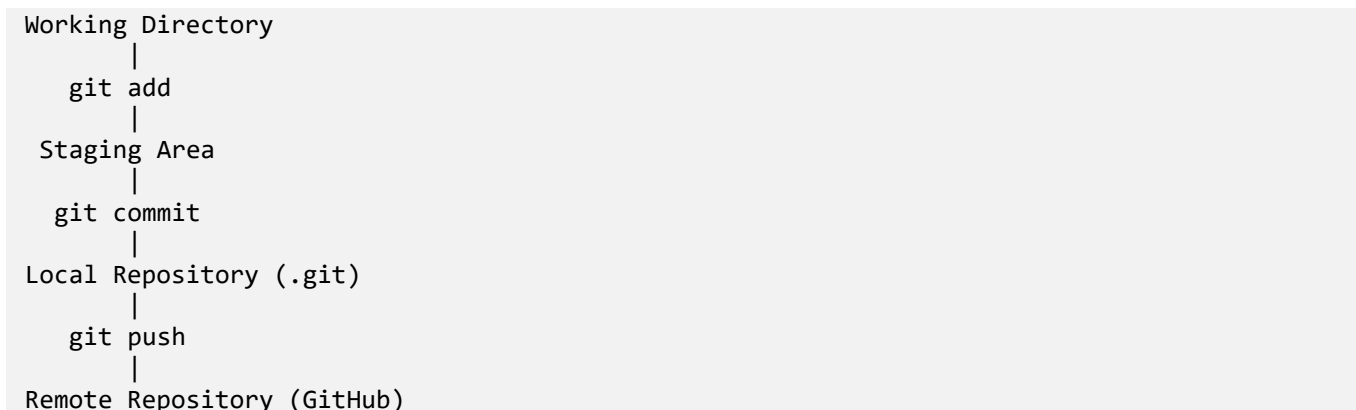
Why is it Needed?

- Enables developers to record and manage changes to their code.
- Provides visibility into the history and status of a project.
- Essential for everyday use of Git in real projects.
- Forms the basic toolkit every developer must know.

Key Points

- `git init` initializes a new Git repository.
- `git status` shows the current state of the working directory (modified, staged, untracked files).
- `git add` stages changes to be included in the next commit.
- `git commit` saves the staged changes as a snapshot with a message.
- `git log` displays the commit history.
- `git diff` shows the differences between file versions.

Diagram / Flowchart



Syntax

<code>git init</code>	# Initialize repository
<code>git status</code>	# Check current status
<code>git add filename.txt</code>	# Stage a specific file
<code>git add .</code>	# Stage all changes
<code>git commit -m "message"</code>	# Commit staged changes
<code>git log</code>	# View commit history
<code>git diff</code>	# View unstaged differences

Example Program: A Typical Git Command Workflow

```
# Typical Git command workflow for a small change
git status
```

```
git add index.html
git commit -m "Add homepage layout"
git log
```

Output

```
On branch main
Changes not staged for commit: index.html

[main 4f9a2c1] Add homepage layout
1 file changed, 10 insertions(+)

commit 4f9a2c1 ... Add homepage layout
```

Real-World Example

In a library management system project, a developer uses git status to check which files were modified, git add and git commit to save the changes with a clear message, and git log to review the entire project history before a release.

Advantages

- Provides precise control over what changes get saved.
- Maintains a clear, readable history of the project through commit messages.
- Helps in reviewing exactly what was changed using git diff.
- Essential and standardized across all Git-based workflows.

Exam Focus

- git add stages changes; git commit saves them permanently with a message.
- git status shows modified, staged, and untracked files.
- git log displays the commit history.
- git diff shows the actual line-by-line differences.

Examination Questions

1-Mark Questions:

76. What does git status do?
77. What does git add do?
78. What does git commit do?
79. What does git log display?

2-Mark Questions:

56. Explain the purpose of git add and git commit together.
57. Differentiate between git status and git diff.
58. What information does git log provide?

4-Mark Questions:

39. Explain the Git workflow using git init, add, commit, and log with a diagram and example.
40. Discuss the purpose and syntax of any four important Git commands.

Unit II: Introduction to React & JSX

Module 1: Introduction to React

Unit II: Introduction to React & JSX

Definition

React is a JavaScript library used for building user interfaces, mainly for single-page applications, developed and maintained by Meta (Facebook).

It uses a component-based architecture and a Virtual DOM to build fast, interactive UIs.

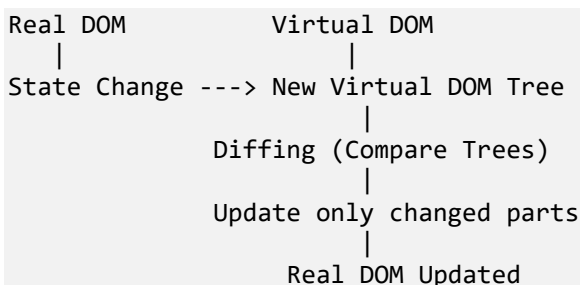
Why is it Needed?

- Building complex UIs with plain JavaScript and direct DOM manipulation is slow and hard to maintain.
- Applications need reusable components to avoid repeating UI code.
- Applications need fast UI updates without reloading the entire page.
- Complex applications need better structure for managing dynamic data (state).
- A large community and ecosystem support faster development.

Key Points

- Developed by Facebook (Jordan Walke); first released in 2013.
- Follows a component-based architecture.
- Uses a Virtual DOM to optimize rendering performance.
- Follows a declarative programming style.
- Follows unidirectional (one-way) data flow.
- Supports both web development (ReactDOM) and mobile development (React Native).
- React is a library, not a full framework.

Diagram / Flowchart



Syntax

```
// Basic React element (without JSX)
const element = React.createElement('h1', null, 'Hello React');

// Same thing using JSX
const element = <h1>Hello React</h1>;
```

Example Program: A Simple React Component

```
// App.jsx
function App() {
```

```
// Simple functional component
return (
  <h1>Welcome to React</h1>
);
}

export default App;
```

Output

Displays "Welcome to React" as a heading in the browser.

Real-World Example

Companies such as Facebook, Instagram, Netflix, and Airbnb use React to build fast, interactive UIs, such as a social media feed that updates instantly without reloading the page.

Advantages

- Fast rendering using the Virtual DOM.
- Reusable components save development time.
- Large community and rich ecosystem of tools and libraries.
- Relatively easy to learn for developers who already know JavaScript.
- Actively maintained by Meta (Facebook).
- Works well with SEO when combined with server-side rendering frameworks such as Next.js.

Limitations

- The fast pace of updates requires developers to keep learning continuously.
- JSX has a learning curve for complete beginners.
- React only handles the UI layer; other libraries are needed for routing and advanced state management.

Exam Focus

- React is a library, not a framework.
- Created by Facebook; first released in 2013.
- Uses the Virtual DOM for performance.
- Component-based and declarative.
- Follows one-way (unidirectional) data flow.

Examination Questions

1-Mark Questions:

80. What is React?
81. Who developed React?
82. In which year was React released?
83. Define Virtual DOM.
84. Is React a library or a framework?
85. What is component-based architecture?

2-Mark Questions:

59. Differentiate between the Real DOM and the Virtual DOM.
60. List any four features of React.

61. Why is React called declarative?

62. What is meant by one-way data binding in React?

4-Mark Questions:

41. Explain what React is and why it is used, with a diagram of how the Virtual DOM works.

42. Discuss the advantages and limitations of React.

43. Explain the features of React with suitable examples.

Module 2: React Development Environment

Unit II: Introduction to React & JSX

Definition

Setting up a React development environment involves installing Node.js and npm, then using a build tool such as Vite to create and run a React project.

Why is it Needed?

- Node.js and npm are required to manage packages and dependencies.
- A bundler and development server are needed for fast development with hot reload.
- An organized project structure is needed for scalability.
- Build tools automate tasks such as converting JSX into plain JavaScript and bundling files.

Key Points

- Node.js is the JavaScript runtime needed to run npm and build tools.
- npm (Node Package Manager) is used to install React and other libraries.
- Vite is a modern, fast build tool, now generally recommended over Create React App.
- create-react-app (CRA) is an older tool, slower, and less commonly used today.
- A new project is created using the command: `npm create vite@latest`.
- Default project structure includes the `src/` folder, `public/` folder, `package.json`, and `index.html`.

Diagram / Flowchart

```
Node.js Installed
|
npm available
|
npm create vite@latest my-app
|
cd my-app --> npm install
|
npm run dev
|
React App Running (localhost)
```

Syntax

```
# Check Node & npm version
node -v
npm -v

# Create new React project using Vite
npm create vite@latest my-app -- --template react

# Move into project folder
cd my-app

# Install dependencies
npm install

# Run development server
```

```
npm run dev
```

Example Program: Default Vite + React Project Structure

```
my-app/  
├── node_modules/  
├── public/  
├── src/  
│   ├── App.jsx  
│   ├── main.jsx  
│   └── index.css  
├── index.html  
├── package.json  
└── vite.config.js
```

Output

Running `npm run dev` starts a local server (for example `http://localhost:5173`) showing the default Vite + React starter page.

Real-World Example

Just as a chef needs a clean, organized kitchen with the right tools before cooking, a developer needs Node.js, npm, and a bundler set up before building a React application, such as an online food delivery app.

Advantages

- Fast development server with Hot Module Replacement (HMR).
- Organized default project structure.
- Easy dependency management using npm.
- Vite offers a much faster build and start time than CRA.
- Easy to add additional libraries, such as React Router or Axios.

Exam Focus

- Node.js and npm are prerequisites for React development.
- Vite is the modern, recommended tool over CRA.
- The command to create a new app is: `npm create vite@latest`.
- The command to run the app is: `npm run dev`.

Examination Questions

1-Mark Questions:

86. What is Node.js?
87. What is npm?
88. Name a tool used to create a React project.
89. What is Vite?
90. Which command runs a React development server?

2-Mark Questions:

63. Differentiate between Vite and Create React App.
64. List the steps to create a new React project using Vite.
65. What is the purpose of package.json?

4-Mark Questions:

44. Explain the steps to set up a React development environment with syntax.
45. Describe the default folder structure of a React project.

Module 3: JSX

Unit II: Introduction to React & JSX

Definition

JSX (JavaScript XML) is a syntax extension for JavaScript that lets developers write HTML-like code inside JavaScript to describe the UI.

Why is it Needed?

- Makes UI code more readable and easier to write.
- Combines markup and logic in one place.
- Compiles down to `React.createElement()` calls.
- Makes it easier to visualize component structure compared to plain JavaScript.

Key Points

- JSX must return a single parent element, or a Fragment.
- Curly braces `{}` are used to embed JavaScript expressions.
- `className` is used instead of `class` to set CSS classes.
- Self-closing tags are required for empty elements, e.g. `<img />`.
- JSX is compiled into plain JavaScript by Babel.
- Attributes use camelCase, such as `onClick` and `tabIndex`.

Diagram / Flowchart

```
JSX Code
|
Babel Compiler
|
React.createElement() calls
|
Virtual DOM Objects
|
Real DOM
```

Syntax

```
const name = "Ravi";

// JSX with embedded expression
const element = <h1>Hello, {name}!</h1>;

// JSX with attributes
const image = ;
```

Example Program: Displaying a Student Report

```
// App.jsx
function App() {
  const studentName = "Asha";
  const marks = 88;

  return (
```

```

    <div>
      <h2>Student Report</h2>
      <p>Name: {studentName}</p>
      <p>Marks: {marks}</p>
    </div>
  );
}

```

```
export default App;
```

Output

```

Student Report
Name: Asha
Marks: 88

```

Real-World Example

Similar to filling a form template (structure) with actual data (JavaScript values), a student management system uses JSX to show each student's name and marks dynamically inside a fixed layout.

Advantages

- Easy to read and write UI code.
- Combines HTML structure and JavaScript logic together.
- Errors are caught at compile time.
- Improves code organization for components.

Limitations

- Browsers cannot read JSX directly; it needs to be compiled first (usually by Babel).
- Slightly different syntax rules, such as `className` and `camelCase` attributes, can confuse beginners.

Exam Focus

- JSX stands for JavaScript XML.
- JSX must return a single root element.
- JSX is compiled to `React.createElement()` via Babel.
- `className` is used instead of `class` in JSX.
- Curly braces `{}` are used to embed JavaScript expressions.

Examination Questions

1-Mark Questions:

91. What does JSX stand for?
92. Which tool compiles JSX to JavaScript?
93. What is used to embed JavaScript inside JSX?
94. Can JSX return multiple root elements directly?

2-Mark Questions:

66. List any four rules of writing JSX.
67. Why is `className` used instead of `class` in JSX?
68. Differentiate between JSX and HTML.

4-Mark Questions:

46. Explain JSX with syntax rules and a suitable example.
47. Explain how JSX is converted into Virtual DOM elements, with a diagram.

Module 4: React Components

Unit II: Introduction to React & JSX

Definition

A component is an independent, reusable block of code that returns JSX to describe part of a user interface. Functional components are plain JavaScript functions that return JSX.

Why is it Needed?

- Breaks the UI into small, manageable, independent pieces.
- Allows code to be reused across the application.
- Makes debugging and testing easier.
- Encourages separation of concerns in application design.

Key Points

- There are two types of components: Functional Components (modern) and Class Components (older).
- A component's name must start with an uppercase letter.
- A component must return a single JSX element, or a Fragment.
- Components can be exported from one file and imported into another.
- Functional components can use Hooks such as `useState` and `useEffect`.

Diagram / Flowchart

```
App Component
|
+-- Header Component
+-- Body Component
|   +-- Card Component
+-- Footer Component
```

Syntax

```
// Defining a functional component
function ComponentName() {
  return <h1>Hello</h1>;
}

// Exporting
export default ComponentName;

// Importing in another file
import ComponentName from './ComponentName';
```

Example Program: A Reusable Greeting Component

```
// Greeting.jsx
function Greeting() {
  // Returns a simple JSX greeting message
  return <h2>Good Morning!</h2>;
}

export default Greeting;
```

```
// App.jsx
import Greeting from './Greeting';

function App() {
  return (
    <div>
      <Greeting />
    </div>
  );
}

export default App;
```

Output

Good Morning!

Real-World Example

Like building a house with prefabricated modules such as a door, window, or roof, React components (Header, Sidebar, Footer) are built once and reused across pages — for example, a banking system's Navbar component reused on every page.

Advantages

- Reusable across multiple parts of an application.
- Easier to maintain and test individually.
- Improves overall code readability.
- Encourages modular application design.

Limitations

- Too many small, poorly organized components can complicate the overall project structure.

Exam Focus

- Component names must start with a capital letter.
- Functional components are plain JavaScript functions that return JSX.
- Components must be exported and imported to be used across files.
- Only one root element can be returned from a component's JSX.

Examination Questions

1-Mark Questions:

95. What is a React component?
96. What are the two types of components in React?
97. What naming convention is used for component names?
98. What must a component return?

2-Mark Questions:

69. Differentiate between functional and class components.
70. Explain how to export and import a component.
71. Why are components considered reusable?

4-Mark Questions:

48. Explain React components with syntax, diagram, and example.
49. Discuss the advantages of using components in React applications.

Module 5: Reusable Components

Unit II: Introduction to React & JSX

Definition

Reusable components are generic components designed using props so that the same component can be used multiple times with different data, avoiding code duplication.

Why is it Needed?

- Avoids rewriting similar code repeatedly.
- Maintains consistency in UI design across the application.
- Makes it easier to update the UI in one place.
- Supports component composition, where complex UIs are built from small components.

Key Points

- Reusability is achieved mainly by using props to customize behaviour and appearance.
- Encourages the DRY (Don't Repeat Yourself) principle.
- Reusable components can be nested inside other components, known as composition.
- Common examples include Button, Card, and InputField components.

Diagram / Flowchart

```
<Card title="A" />
<Card title="B" />
<Card title="C" />
  |
Same Card Component
rendered 3 times with
different props
```

Syntax

```
// Reusable component using props
function Card(props) {
  return <div className="card">{props.title}</div>;
}

// Using the same component multiple times
<Card title="Math" />
<Card title="Science" />
```

Example Program: A Reusable Subject Card

```
// Card.jsx
function Card(props) {
  // Reusable card that displays any subject name
  return (
    <div className="card">
      <h3>{props.subject}</h3>
    </div>
  );
}
```

```
export default Card;

// App.jsx
import Card from './Card';

function App() {
  return (
    <div>
      <Card subject="Mathematics" />
      <Card subject="Physics" />
      <Card subject="Computer Science" />
    </div>
  );
}

export default App;
```

Output

```
Mathematics
Physics
Computer Science
```

Real-World Example

An online food delivery app reuses the same "Restaurant Card" component to display hundreds of different restaurants, simply changing the name, image, and rating each time.

Advantages

- Reduces code duplication significantly.
- Speeds up overall development.
- Easier maintenance, since a change in one place reflects everywhere.
- Promotes a consistent UI and user experience.

Exam Focus

- Reusability is achieved mainly through props.
- Follows the DRY (Don't Repeat Yourself) principle.
- Component composition means building UI from smaller reusable pieces.

Examination Questions

1-Mark Questions:

99. What makes a React component reusable?
100. Expand the DRY principle.
101. Give one example of a reusable component.

2-Mark Questions:

72. Explain component composition with an example.
73. How do props help in creating reusable components?

4-Mark Questions:

50. Explain reusable components with syntax and a suitable example.
51. Discuss the advantages of building reusable components in a real-world application.

Module 6: Props

Unit II: Introduction to React & JSX

Definition

Props (short for Properties) are read-only inputs passed from a parent component to a child component to customize its content or behaviour.

Why is it Needed?

- Enables communication between components, from parent to child.
- Makes components dynamic and configurable.
- Supports reusable component design.
- Keeps the data flow predictable, since it only moves in one direction.

Key Points

- Props are read-only (immutable) inside the child component.
- Props are passed like HTML attributes, e.g. `<Child name="Ravi" />`.
- Props are accessed in a functional component via the props parameter, or through destructuring.
- Data flows only one way: from parent to child.
- Props can carry strings, numbers, arrays, objects, or even functions.

Diagram / Flowchart

```
Parent Component
  | props (data)
  v
Child Component
(receives & displays props)
```

Syntax

```
// Passing props
<Student name="Riya" age={20} />

// Receiving props in a child component
function Student(props) {
  return <p>{props.name} is {props.age} years old</p>;
}

// Using destructuring
function Student({ name, age }) {
  return <p>{name} is {age} years old</p>;
}
```

Example Program: Displaying Student Marks Using Props

```
// Student.jsx
function Student(props) {
  // Displays student details passed from parent
  return (
    <p>{props.name} scored {props.marks} marks</p>
  );
}
```

```
export default Student;

// App.jsx
import Student from './Student';

function App() {
  return (
    <div>
      <Student name="Anil" marks={85} />
      <Student name="Divya" marks={92} />
    </div>
  );
}

export default App;
```

Output

```
Anil scored 85 marks
Divya scored 92 marks
```

Real-World Example

In a student management system, the same "StudentCard" component receives a different name and marks as props for every student record, instead of creating a new component for each student.

Advantages

- Enables dynamic and reusable components.
- Simplifies passing data from parent to child.
- Improves code readability and modularity.
- Encourages predictable, one-way data flow.

Limitations

- Props cannot be modified by the child component, since they are read-only.
- Passing props through many nested components, known as prop drilling, can become complex.

Exam Focus

- Props are read-only (immutable).
- Data flows only from parent to child (one-way).
- Props are passed like HTML attributes.
- "Prop drilling" is a common exam term.

Examination Questions

1-Mark Questions:

102. What are props in React?
103. Are props mutable or immutable?
104. In which direction does data flow when using props?
105. Define prop drilling.

2-Mark Questions:

74. How are props passed and accessed in a functional component?

75. Differentiate between props and state.

76. Give an example of passing multiple props.

4-Mark Questions:

52. Explain props with syntax, diagram, and an example program.

53. Discuss the advantages and limitations of using props in React.

Module 7: Conditional Rendering

Unit II: Introduction to React & JSX

Definition

Conditional rendering in React refers to displaying different UI elements or components based on certain conditions, using JavaScript logic such as if statements, ternary operators, or logical operators inside JSX.

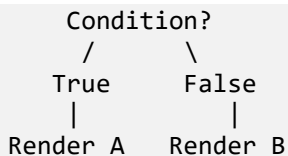
Why is it Needed?

- UIs often need to show different content based on data or user state, such as logged in versus logged out.
- Avoids rendering unnecessary elements, improving performance.
- Makes the UI dynamic and responsive to changing data.
- Essential for showing loading states, error messages, or empty states.

Key Points

- An if statement can be used outside JSX to decide what a component returns.
- The ternary operator (condition ? true : false) is used inside JSX for inline conditions.
- The logical AND (&&) operator renders an element only if a condition is true.
- Rendering lists uses the .map() method to loop through array data and return JSX for each item.
- A key prop is required when rendering lists, helping React track elements efficiently.

Diagram / Flowchart



Syntax

```
// Ternary operator
{isLoggedIn ? <p>Welcome back!</p> : <p>Please log in</p>}

// Logical AND
{hasNotification && <p>You have new messages</p>}

// Rendering a list
{items.map(item => <li key={item.id}>{item.name}</li>)}
```

Example Program: Welcome Message and Student List

```
// App.jsx
function App() {
  const isLoggedIn = true;
  const students = ["Riya", "Karan", "Asha"];

  return (
    <div>
      {/* Ternary for conditional message */}
      {isLoggedIn ? <h2>Welcome back!</h2> : <h2>Please log in</h2>}

      {/* Rendering a list using map() */}
```

```

    <ul>
      {students.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

Output

```

Welcome back!
- Riya
- Karan
- Asha

```

Real-World Example

An e-commerce website conditionally displays "Add to Cart" or "Out of Stock" based on product availability, and renders the entire product list dynamically using `.map()`.

Advantages

- Makes the UI dynamic based on data or user actions.
- Prevents rendering of unnecessary elements.
- Improves user experience with relevant, up-to-date content.
- Simplifies list rendering with the `map()` method.

Limitations

- Overusing nested ternary operators can reduce code readability.
- Missing the key prop when rendering lists can cause performance warnings and bugs.

Exam Focus

- Three main approaches: if statement, ternary operator, and logical AND (`&&`).
- `map()` is used for rendering lists in React.
- The key prop is mandatory and must be unique for list items.
- Ternary operator format: `condition ? valueIfTrue : valueIfFalse`.

Examination Questions

1-Mark Questions:

106. What is conditional rendering?
107. Name any two ways to perform conditional rendering in React.
108. Which method is used to render a list of items?
109. Why is the key prop important?

2-Mark Questions:

77. Explain the ternary operator with an example in JSX.
78. Explain the logical AND (`&&`) operator for conditional rendering.
79. What happens if the key prop is missing while rendering a list?

4-Mark Questions:

54. Explain conditional rendering with syntax and example programs for if, ternary, and &&.
55. Explain how to render lists in React using map() with a suitable example.

Module 8: Event Handling in React

Unit II: Introduction to React & JSX

Definition

Event handling in React refers to responding to user actions such as clicks, input changes, and form submissions, using event handler functions attached directly to JSX elements.

Why is it Needed?

- Enables interactivity within React components.
- Allows components to respond to user input and update the UI.
- Essential for handling forms, buttons, and other interactive elements.
- Works closely with state to create dynamic, responsive interfaces.

Key Points

- Event handlers are written in camelCase, e.g. `onClick`, `onChange`, `onSubmit`.
- Event handler functions are passed as references, not called directly — `{handleClick}`, not `{handleClick()}`.
- React wraps native browser events in a cross-browser-consistent `SyntheticEvent` object.
- `event.preventDefault()` prevents default browser behaviour, such as a form submission reloading the page.
- Inline arrow functions can be used to pass arguments to event handlers.

Diagram / Flowchart

```
graph TD
    A[User Clicks / Types] --> B[React Detects Event (onClick, onChange)]
    B --> C[Event Handler Function Executes]
    C --> D[State Updates / UI Re-renders]
```

Syntax

```
// Handling a click event
function handleClick() {
  console.log("Button clicked!");
}

<button onClick={handleClick}>Click Me</button>

// Handling input change
<input onChange={(e) => console.log(e.target.value)} />

// Handling form submit
<form onSubmit={(e) => {
  e.preventDefault();
  console.log("Form submitted");
}}>
```

Example Program: Handling a Login Button Click

```
// LoginForm.jsx
function LoginForm() {
```

```
// Handles click event on the login button
function handleLogin() {
  alert("Login button clicked!");
}

return (
  <div>
    <button onClick={handleLogin}>Login</button>
  </div>
);
}

export default LoginForm;
```

Output

Clicking the "Login" button displays an alert saying "Login button clicked!".

Real-World Example

A banking application uses `onClick` to handle the "Transfer Funds" button and `onChange` to instantly validate the amount entered by the user before submission.

Advantages

- Makes React components interactive and responsive.
- Provides consistent behaviour across browsers using `SyntheticEvent`.
- Works seamlessly with state to build dynamic interfaces.
- Simple, declarative syntax directly inside JSX.

Limitations

- Passing inline functions in JSX can affect performance in large lists if not optimized.
- Beginners often mistakenly call the function instead of passing a reference, e.g. `onClick={handleClick()}` instead of `onClick={handleClick}`.

Exam Focus

- Event handlers use camelCase naming, such as `onClick` and `onChange`.
- A function reference is passed to an event handler, not invoked directly.
- `SyntheticEvent` is React's cross-browser event wrapper.
- `preventDefault()` stops the default browser action.

Examination Questions

1-Mark Questions:

110. Name any two event handlers used in React.
111. What is a `SyntheticEvent`?
112. What does `event.preventDefault()` do?
113. How are event handler names written in React?

2-Mark Questions:

80. Explain how to handle a click event in React with an example.
81. Differentiate between calling a function and passing a function reference in an event handler.

82. Explain the purpose of the onChange event.

4-Mark Questions:

56. Explain event handling in React with syntax and a suitable example.

57. Discuss common mistakes made while handling events in React with corrected examples.

Module 9: Component Best Practices

Unit II: Introduction to React & JSX

Definition

Component best practices are recommended guidelines for organizing, naming, and structuring React components to make applications maintainable, readable, and scalable.

Why is it Needed?

- Ensures consistency across a growing codebase.
- Makes it easier for teams to collaborate on the same project.
- Improves code readability and reduces bugs.
- Prepares projects to scale smoothly as more features are added.

Key Points

- Organize components into a clear folder structure, e.g. components/, pages/, hooks/.
- Use PascalCase for both component names and file names, e.g. StudentCard.jsx.
- Keep components small and focused on a single responsibility.
- Reuse components wherever possible instead of duplicating code.
- Separate logic and UI where appropriate, e.g. by using custom hooks for logic.
- Use meaningful, descriptive names for props and variables.

Diagram / Flowchart

```
src/  
+-- components/  
|   +-- Header.jsx  
|   +-- Footer.jsx  
|   +-- Card.jsx  
+-- pages/  
|   +-- HomePage.jsx  
+-- hooks/  
    +-- useFetchData.js
```

Syntax

```
// Recommended folder structure (conceptual)  
src/  
  components/  
  pages/  
  hooks/  
  assets/
```

Example Program: A Small, Focused, Reusable Component

```
// Button.jsx  
// Good practice: small, focused, reusable component  
function Button({ label, onClick }) {  
  // Single responsibility: render a styled button  
  return <button className="btn" onClick={onClick}>{label}</button>;  
}
```

```
export default Button;
```

Output

A reusable Button component can now be used anywhere in the app with different labels and click handlers.

Real-World Example

In a large hospital management system with many developers, following consistent naming conventions and folder structures, such as components/ and pages/, helps new team members quickly understand and navigate the codebase.

Advantages

- Improves maintainability of large codebases.
- Makes onboarding new developers easier.
- Reduces bugs caused by inconsistent structure.
- Encourages reusability and modular design.

Exam Focus

- PascalCase is used for component file and function names.
- A folder structure with components/, pages/, and hooks/ is commonly recommended.
- The Single Responsibility Principle states each component should do one thing well.
- Reusability and meaningful naming are frequently tested concepts.

Examination Questions

1-Mark Questions:

114. What naming convention is used for React component names?
115. Name any two folders commonly used in a React project structure.
116. What is the Single Responsibility Principle?

2-Mark Questions:

83. List any four best practices for organizing React components.
84. Why is a consistent folder structure important in team projects?
85. Explain the benefit of keeping components small and focused.

4-Mark Questions:

58. Discuss best practices for structuring and naming React components with examples.
59. Explain how following best practices improves maintainability in large-scale React applications.

Module 10: React Hooks

Unit II: Introduction to React & JSX

Definition

Hooks are special functions introduced in React 16.8 that let functional components use features such as state and lifecycle behaviour, which were previously only available in class components.

Why is it Needed?

- Allows functional components to manage state and side effects without converting to class components.
- Simplifies code by removing the need for complex class syntax.
- Encourages logic reuse through custom hooks.
- Makes React code shorter, cleaner, and easier to understand.

Key Points

- Hooks always start with the word "use", e.g. useState, useEffect.
- Rule of Hooks: only call hooks at the top level, not inside loops, conditions, or nested functions.
- Rule of Hooks: only call hooks from React function components or custom hooks.
- Common built-in hooks: useState, useEffect, useContext, useRef.
- Hooks enable functional components to fully replace class components in most cases.

Diagram / Flowchart

```
Functional Component
|
|  useState() --> manages state
|  useEffect() --> handles side effects
|  useContext()--> accesses context
|
|  Component re-renders as needed
```

Syntax

```
import { useState, useEffect } from 'react';

function MyComponent() {
  const [count, setCount] = useState(0); // useState hook

  useEffect(() => {
    console.log("Component rendered");
  }, []); // useEffect hook

  return <p>{count}</p>;
}
```

Example Program: A Simple Counter Using useState

```
// Counter.jsx
import { useState } from 'react';

function Counter() {
  // useState hook manages the count value
  const [count, setCount] = useState(0);
```

```

    return (
      <div>
        <p>Count: {count}</p>
        <button onClick={() => setCount(count + 1)}>Increment</button>
      </div>
    );
  }
}

export default Counter;

```

Output

Displays "Count: 0" initially; clicking "Increment" increases the count by 1 each time.

Real-World Example

In a food delivery app, a hook like `useState` manages the quantity of an item added to the cart, updating instantly as the user increases or decreases it.

Advantages

- Simplifies state and lifecycle management in functional components.
- Removes the need for complex class-based syntax.
- Encourages reusable logic through custom hooks.
- Leads to shorter, more readable component code.

Limitations

- Rules of Hooks must be strictly followed, which can confuse beginners.
- Overusing multiple hooks in one component can reduce readability if not organized well.

Exam Focus

- Hooks were introduced in React 16.8.
- Hook names always start with "use".
- Two Rules of Hooks: call at the top level only; call only from React functions.
- `useState` and `useEffect` are the most commonly used hooks.

Examination Questions

1-Mark Questions:

117. What are React Hooks?
118. In which React version were Hooks introduced?
119. Name any two built-in React Hooks.
120. What word do all hook names begin with?

2-Mark Questions:

86. Explain the Rules of Hooks.
87. Why were Hooks introduced in React?
88. Differentiate between class components and functional components using Hooks.

4-Mark Questions:

60. Explain React Hooks with syntax and an example program.

61. Discuss the advantages of using Hooks over class components.

Module 11: State Management

Unit II: Introduction to React & JSX

Definition

State refers to data that is managed within a component and can change over time, causing the component to re-render whenever it is updated.

Why is it Needed?

- Allows components to store and manage dynamic data, such as user input or toggled UI states.
- Enables the UI to automatically update when the underlying data changes.
- Essential for building interactive features like counters, forms, and toggles.
- Differentiates dynamic components from purely static ones.

Key Points

- State is local to a component unless it is explicitly shared, e.g. via props or the Context API.
- Unlike props, state is mutable and can be changed within its own component.
- Updating state triggers a re-render of the component.
- State should never be modified directly; it must always be updated using a setter function.
- React may batch multiple state updates together for performance in many cases.

Diagram / Flowchart

```
graph TD
    A[Initial State] --> B[User Action (e.g. click)]
    B --> C[setState() called]
    C --> D[State Updated]
    D --> E[Component Re-renders]
```

Syntax

```
// Conceptual state flow (implemented via the useState hook)
let state = initialValue;

function updateState(newValue) {
  state = newValue;
  // Component re-renders automatically
}
```

Example Program: Toggling a Message On and Off

```
// ToggleMessage.jsx
import { useState } from 'react';

function ToggleMessage() {
  const [visible, setVisible] = useState(true);

  return (
    <div>
```

```

    {visible && <p>Hello, this message can be toggled!</p>}
    <button onClick={() => setVisible(!visible)}>Toggle</button>
  </div>
);
}

export default ToggleMessage;

```

Output

Clicking "Toggle" repeatedly shows and hides the message.

Real-World Example

In a social media application, whether a post's "like" button appears filled or outlined depends on the component's state, which updates instantly when the user clicks it.

Advantages

- Enables components to be dynamic and interactive.
- Automatically re-renders the UI when data changes.
- Keeps data management localized and predictable within components.
- Forms the foundation for building complex, stateful applications.

Limitations

- Directly mutating state, instead of using the setter function, can lead to unpredictable bugs.
- Managing state across many components can become complex without additional tools, such as the Context API.

Exam Focus

- State is mutable; props are immutable.
- State updates trigger component re-renders.
- State must be updated using a setter function, never directly.
- The difference between props and state is one of the most frequently asked questions.

Examination Questions

1-Mark Questions:

121. What is state in React?
122. Is state mutable or immutable?
123. What happens when state is updated?
124. How should state be updated in React?

2-Mark Questions:

89. Differentiate between props and state.
90. Why should state never be modified directly?
91. Explain why updating state causes a component to re-render.

4-Mark Questions:

62. Explain the concept of state in React with a diagram and example.
63. Discuss why state management is essential in building interactive React applications.

Module 12: useState Hook

Unit II: Introduction to React & JSX

Definition

useState is a built-in React Hook that allows functional components to create and manage state variables, returning the current state value and a function to update it.

Why is it Needed?

- Enables functional components to hold and manage local state.
- Provides a simple way to trigger re-renders when data changes.
- Removes the need for class components just to use state.
- Essential for building interactive elements like counters, forms, and toggles.

Key Points

- Syntax: `const [state, setState] = useState(initialValue);`
- Returns an array with two elements: the current state and a setter function.
- Calling the setter function updates the state and triggers a re-render.
- Multiple state variables can be declared using multiple useState calls.
- State updates are asynchronous and may be batched by React.
- The initial value passed to useState can be a primitive, array, object, or a function result.

Diagram / Flowchart

```
useState(initialValue)
  |
Returns [state, setState]
  |
setState(newValue) called
  |
Component re-renders with new state
```

Syntax

```
import { useState } from 'react';

const [count, setCount] = useState(0);

// Updating state
setCount(count + 1);

// Multiple state variables
const [name, setName] = useState("");
const [age, setAge] = useState(0);
```

Example Program: An Increment/Decrement Counter

```
// Counter.jsx
import { useState } from 'react';

function Counter() {
  // Creating a state variable "count" with initial value 0
```

```

const [count, setCount] = useState(0);

return (
  <div>
    <p>Current Count: {count}</p>
    <button onClick={() => setCount(count + 1)}>Increase</button>
    <button onClick={() => setCount(count - 1)}>Decrease</button>
  </div>
);
}

export default Counter;

```

Output

Displays "Current Count: 0"; clicking "Increase" or "Decrease" updates the count accordingly.

Real-World Example

An online food delivery app uses `useState` to manage the quantity of an item in the cart, updating the displayed value instantly as the user taps the "+" or "-" buttons.

Advantages

- Simple and easy-to-use syntax for managing state.
- Enables functional components to become fully interactive.
- Supports multiple independent state variables in a single component.
- Automatically triggers re-renders when state changes.

Limitations

- Directly modifying the state variable without using the setter function will not trigger a re-render.
- Managing many separate `useState` calls in one component can become harder to organize; an object or reducer may be better for complex state.

Exam Focus

- `useState` returns an array: `[state, setterFunction]`.
- State updates using the setter function trigger re-renders.
- `useState` can hold any data type as its initial value.
- `useState` is one of the most frequently tested Hooks in exams.

Examination Questions

1-Mark Questions:

125. What does `useState` return?
126. Write the basic syntax of the `useState` Hook.
127. Can a component have multiple `useState` calls?
128. What triggers a component to re-render when using `useState`?

2-Mark Questions:

92. Explain the `useState` Hook with a simple example.
93. What happens if state is modified without using the setter function?
94. List the two values returned by `useState`.

4-Mark Questions:

64. Explain the useState Hook with syntax, diagram, and an example program.
65. Write a React program using useState to build a simple counter and explain each part.

Module 13: useEffect Hook

Unit II: Introduction to React & JSX

Definition

useEffect is a built-in React Hook that lets functional components perform side effects, such as fetching data, updating the DOM, or setting up subscriptions, after rendering.

Why is it Needed?

- Enables handling of side effects, such as API calls, timers, and subscriptions, in functional components.
- Replaces lifecycle methods like componentDidMount, componentDidUpdate, and componentWillUnmount used in class components.
- Keeps side-effect logic separate from the main rendering logic.
- Essential for tasks like fetching data from a server when a component loads.

Key Points

- Basic syntax: `useEffect(() => { /* effect */ }, [dependencies]);`
- The effect runs after the component renders.
- The dependency array controls when the effect runs: `[]` runs once on mount, `[value]` runs when value changes, and no array runs on every render.
- A cleanup function, returned from `useEffect`, runs before the component unmounts or before the effect re-runs.
- Commonly used for API calls, event listeners, timers, and subscriptions.

Diagram / Flowchart

```
Component Renders
|
useEffect Runs (after render)
|
Side Effect Executes (API call, etc.)
|
Cleanup Function Runs (before next effect or unmount)
```

Syntax

```
import { useEffect } from 'react';

useEffect(() => {
  // Code to run after render
  console.log("Effect ran");

  return () => {
    // Cleanup code (optional)
    console.log("Cleanup before next effect or unmount");
  };
}, [dependency]);
```

Example Program: Fetching User Data on Mount

```
// UserData.jsx
import { useState, useEffect } from 'react';
```

```
function UserData() {
  const [user, setUser] = useState(null);

  // Runs once when the component mounts
  useEffect(() => {
    fetch("https://api.example.com/user/1")
      .then(response => response.json())
      .then(data => setUser(data));
  }, []); // empty array = run once

  return <p>{user ? user.name : "Loading..."}</p>;
}

export default UserData;
```

Output

Initially displays "Loading...", then updates to show the fetched user's name once the API call completes.

Real-World Example

A student portal uses `useEffect` to fetch a student's attendance data from the server as soon as the dashboard page loads.

Advantages

- Handles side effects cleanly within functional components.
- The dependency array gives fine control over when effects run.
- The cleanup function helps prevent memory leaks, such as removing event listeners.
- Replaces multiple class lifecycle methods with a single, flexible hook.

Limitations

- Incorrect dependency arrays can cause infinite loops or missed updates.
- Can be confusing for beginners to understand exactly when effects run and re-run.

Exam Focus

- `useEffect` replaces `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.
- An empty dependency array `[]` means the effect runs only once, on mount.
- A cleanup function is returned from inside `useEffect`.
- The dependency array is a very frequently tested exam concept.

Examination Questions

1-Mark Questions:

129. What is the `useEffect` Hook used for?
130. What does an empty dependency array `[]` mean in `useEffect`?
131. Name any one class lifecycle method that `useEffect` replaces.
132. What is a cleanup function in `useEffect`?

2-Mark Questions:

95. Explain the syntax of `useEffect` with an example.

96. Explain the role of the dependency array in `useEffect`.

97. Why is a cleanup function used in `useEffect`?

4-Mark Questions:

66. Explain the `useEffect` Hook with syntax, diagram, and a suitable example.

67. Discuss how the dependency array affects the behaviour of `useEffect` with examples.

Module 14: Custom Hooks

Unit II: Introduction to React & JSX

Definition

A custom hook is a reusable JavaScript function, whose name starts with "use", that combines one or more built-in hooks to encapsulate and share logic between multiple components.

Why is it Needed?

- Avoids duplicating the same stateful logic across multiple components.
- Makes components cleaner by extracting complex logic into separate functions.
- Encourages logic reuse without needing to repeat code.
- Improves testability by isolating logic from the UI.

Key Points

- Custom hook names must start with "use", e.g. useFetchData, useForm.
- Internally, a custom hook can use other hooks, such as useState and useEffect.
- Custom hooks follow the same Rules of Hooks as built-in hooks.
- A custom hook returns values or functions that components can use, just like a built-in hook.
- Custom hooks do not share state between components; each component using it gets its own independent instance.

Diagram / Flowchart



Syntax

```
// Defining a custom hook
function useCounter(initialValue) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(count + 1);

  return { count, increment };
}

// Using the custom hook in a component
const { count, increment } = useCounter(0);
```

Example Program: A Custom Hook for Fetching Data

```
// useFetchData.js - a custom hook
import { useState, useEffect } from 'react';
```

```
function useFetchData(url) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(result => setData(result));
  }, [url]);

  return data;
}

export default useFetchData;

// ProductList.jsx
import useFetchData from './useFetchData';

function ProductList() {
  const products = useFetchData("https://api.example.com/products");
  return <p>{products ? products.length : "Loading..."} products found</p>;
}

export default ProductList;
```

Output

Displays "Loading..." initially, then shows the number of products fetched from the API.

Real-World Example

An online food delivery app might use a custom hook, such as `useFetchRestaurants()`, to fetch and reuse restaurant list logic across multiple pages, such as the home page and the search page.

Advantages

- Promotes reusability of stateful logic across multiple components.
- Keeps components cleaner and more focused on rendering UI.
- Makes complex logic easier to test in isolation.
- Encourages a consistent approach to solving similar problems.

Limitations

- Overusing custom hooks for very simple logic can add unnecessary complexity.
- Each component using a custom hook gets an independent state, which can confuse beginners expecting shared state.

Exam Focus

- Custom hook names must start with "use".
- Custom hooks can combine multiple built-in hooks internally.
- Custom hooks follow the same Rules of Hooks as built-in hooks.
- Each component gets its own independent instance of a custom hook's state.

Examination Questions

1-Mark Questions:

- 133. What is a custom hook?
- 134. What must a custom hook's name always start with?
- 135. Name any one benefit of using custom hooks.
- 136. Do custom hooks share state between components?

2-Mark Questions:

- 98. Explain why custom hooks are used in React.
- 99. Give an example of a custom hook and explain its purpose.
- 100. List any two rules that apply to custom hooks.

4-Mark Questions:

- 68. Explain custom hooks with syntax and a suitable example program.
- 69. Discuss the advantages of creating custom hooks for logic reuse in React applications.

Module 15: React Routing

Unit II: Introduction to React & JSX

Definition

React Routing refers to the mechanism of navigating between different pages or views in a React application without reloading the entire page, typically implemented using the React Router library.

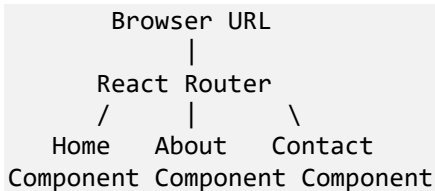
Why is it Needed?

- Enables multi-page-like navigation within a Single Page Application (SPA).
- Improves user experience with fast, seamless page transitions.
- Allows different components to be rendered based on the current URL.
- Essential for building applications with multiple views, such as Home, About, and Contact pages.

Key Points

- Routing maps specific URL paths to specific components.
- Navigation happens without a full page reload, keeping the app fast.
- React itself does not include routing; a library such as React Router is used.
- Each "page" in a React app is simply a component rendered based on the current route.
- Supports nested routes, dynamic routes with parameters, and redirects.

Diagram / Flowchart



Syntax

```
// Conceptual routing setup (implemented using a routing library)
Routes:
"/"          -> Home Component
"/about"     -> About Component
"/contact"   -> Contact Component
```

Example Program: Rendering a Component Based on the Current Path

```
// Conceptual example of route-based rendering
function App({ currentPath }) {
  // Render component based on current URL path
  if (currentPath === "/about") {
    return <About />;
  } else if (currentPath === "/contact") {
    return <Contact />;
  } else {
    return <Home />;
  }
}
```

Output

Depending on the URL path, the Home, About, or Contact component is displayed without reloading the browser.

Real-World Example

A college website uses routing so that navigating from the "Home" page to the "Admissions" page updates the content instantly without a full page reload, giving a smoother user experience.

Advantages

- Provides fast, seamless navigation without full page reloads.
- Improves overall user experience in Single Page Applications.
- Allows clear separation of pages and views into components.
- Supports dynamic and nested routing for complex applications.

Limitations

- Requires an additional library, such as React Router, since React does not include routing by default.
- Improper route configuration can lead to broken links or 404 errors.

Exam Focus

- Routing enables navigation without full page reloads in a Single Page Application (SPA).
- React does not include built-in routing; a library like React Router is required.
- URL paths are mapped to specific components.
- SPA stands for Single Page Application.

Examination Questions

1-Mark Questions:

137. What is React Routing?
138. Expand SPA.
139. Does React include built-in routing?
140. Name the most commonly used routing library in React.

2-Mark Questions:

101. Explain why routing is needed in a React application.
102. What is a Single Page Application?
103. List any two features of React Routing.

4-Mark Questions:

70. Explain the concept of routing in React with a diagram and example.
71. Discuss the advantages of using routing in Single Page Applications.

Module 16: Client-side vs Server-side Routing

Unit II: Introduction to React & JSX

Definition

Client-side routing handles navigation within the browser using JavaScript without contacting the server for each page, while server-side routing requires a new request to the server for every page navigation.

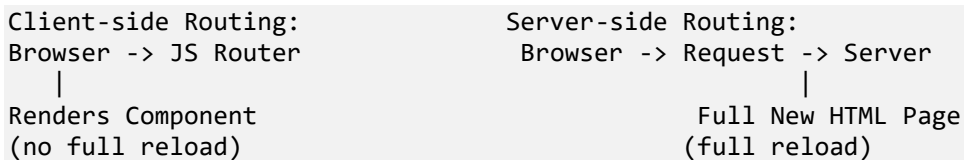
Why is it Needed?

- Understanding both approaches helps developers choose the right architecture for their application.
- Client-side routing improves speed and user experience in modern Single Page Applications.
- Server-side routing remains important for SEO and initial page load performance.
- Many modern frameworks combine both approaches, for example through server-side rendering.

Key Points

- Client-side routing: JavaScript intercepts navigation and renders the appropriate component without a full page reload.
- Server-side routing: each URL request is sent to the server, which returns a new, complete HTML page.
- Client-side routing offers faster transitions after the initial page load.
- Server-side routing is generally better for SEO, since search engines can easily read fully rendered HTML.
- Frameworks such as Next.js combine both approaches using server-side rendering (SSR) and client-side navigation.

Diagram / Flowchart



Syntax

```
// Client-side: React Router intercepts navigation
<Link to="/about">About</Link>

// Server-side: standard anchor tag causes full page reload
<a href="/about.html">About</a>
```

Example Program: Comparing Client-side and Server-side Links

```
// Client-side routing example (conceptual)
// Clicking this link does NOT reload the page
<Link to="/dashboard">Go to Dashboard</Link>

// Server-side routing example (traditional website)
// Clicking this link DOES reload the page
<a href="/dashboard.html">Go to Dashboard</a>
```

Output

The client-side link instantly swaps the visible component; the server-side link causes a full browser page reload.

Real-World Example

In an online banking application, navigating between "Accounts" and "Transactions" tabs uses fast client-side routing, while the very first login page load is often handled by the server, using server-side rendering, for security and performance.

Advantages

- Client-side routing offers faster, smoother navigation within an app.
- Server-side routing offers better SEO and faster initial content visibility.
- Understanding both allows developers to choose the right approach for each use case.
- Modern hybrid approaches, combining SSR with client-side routing, offer the benefits of both.

Limitations

- Pure client-side routing can face SEO challenges without additional configuration.
- Pure server-side routing feels slower for frequent navigation within the same app.

Exam Focus

- Client-side routing avoids full page reloads; server-side routing requires them.
- Server-side routing is generally better for SEO.
- React Router is a client-side routing solution.
- The difference between client-side and server-side routing is a commonly asked comparison question.

Examination Questions

1-Mark Questions:

141. What is client-side routing?
142. What is server-side routing?
143. Which type of routing avoids full page reloads?
144. Which type of routing is generally better for SEO?

2-Mark Questions:

104. Differentiate between client-side and server-side routing.
105. List any two advantages of client-side routing.
106. Give one real-world example of server-side routing.

4-Mark Questions:

72. Explain the differences between client-side and server-side routing with a diagram.
73. Discuss the advantages and limitations of client-side routing compared to server-side routing.

Module 17: React Router

Unit II: Introduction to React & JSX

Definition

React Router is a popular third-party library used to implement client-side routing in React applications, providing components such as BrowserRouter, Routes, Route, Link, and Navigate.

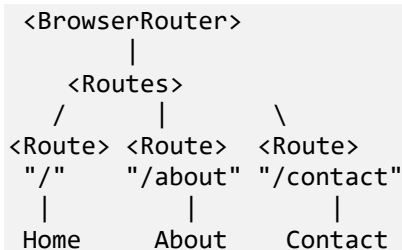
Why is it Needed?

- React does not have built-in routing, so a dedicated library is required.
- Enables developers to define multiple views or pages within a Single Page Application.
- Provides simple, declarative syntax for defining routes and navigation.
- Widely adopted and well-supported for real-world React projects.

Key Points

- BrowserRouter wraps the application and enables routing using the HTML5 history API.
- Routes is a container that holds all individual Route definitions.
- Route defines a mapping between a URL path and a component to render.
- Link is used instead of the anchor <a> tag for client-side navigation without a page reload.
- Navigate is used to programmatically redirect users to a different route.

Diagram / Flowchart



Syntax

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

<BrowserRouter>
  <Link to="/">Home</Link>
  <Link to="/about">About</Link>

  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

Example Program: A Simple App with Two Routes

```
// App.jsx
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}
```

```

}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      { /* Navigation links */}
      <Link to="/">Home</Link> | <Link to="/about">About</Link>

      { /* Route definitions */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

```

Output

Clicking "Home" displays "Home Page"; clicking "About" displays "About Page", without a full page reload.

Real-World Example

A library management system uses React Router to switch between "Catalogue", "My Books", and "Profile" pages instantly as the user clicks the navigation menu.

Advantages

- Enables clean, declarative route definitions.
- Provides fast client-side navigation without page reloads.
- Supports nested routes, dynamic parameters, and redirects.
- Well-documented, with strong community support.

Limitations

- Adds an additional dependency to the project.
- Requires understanding of routing concepts, which can be challenging for complete beginners.

Exam Focus

- React Router provides BrowserRouter, Routes, Route, Link, and Navigate.
- Link is used instead of <a> for client-side navigation.
- Route maps a path to a component using the element prop.
- React Router must be installed separately, as it is not part of core React.

Examination Questions

1-Mark Questions:

145. What is React Router?
146. Name any two components provided by React Router.

- 147. Which component is used instead of the anchor tag for navigation?
- 148. Which component defines a mapping between a path and a component?

2-Mark Questions:

- 107. Explain the purpose of BrowserRouter and Routes.
- 108. Differentiate between the Link component and a normal <a> tag.
- 109. What is the purpose of the Navigate component?

4-Mark Questions:

- 74. Explain React Router with syntax and a suitable example program.
- 75. Discuss the role of BrowserRouter, Routes, Route, and Link with an example.

Module 18: Class Components

Unit II: Introduction to React & JSX

Definition

Class components are React components defined using ES6 classes that extend `React.Component`, capable of holding their own state and lifecycle methods. They were the primary way to manage state before Hooks were introduced.

Why is it Needed?

- Historically the only way to use state and lifecycle methods in React, before Hooks existed.
- Many older React codebases still use class components, so understanding them remains important.
- Helps in understanding the underlying concepts that Hooks were designed to simplify.
- Some legacy libraries and interview questions still reference class components.

Key Points

- A class component extends `React.Component`.
- It must include a `render()` method that returns JSX.
- State is initialized in the constructor using `this.state = {...}`.
- State is updated using `this.setState()`, and should never be modified directly.
- Props are accessed using `this.props`.
- Lifecycle methods, such as `componentDidMount`, are available, unlike in plain functional components without hooks.

Diagram / Flowchart

```
class MyComponent extends React.Component
  |
  constructor() { this.state = {...} }
  |
  render() { return JSX }
  |
  Lifecycle Methods (componentDidMount, etc.)
```

Syntax

```
import React from 'react';

class Welcome extends React.Component {
  constructor(props) {
    super(props);
    this.state = { message: "Hello" };
  }

  render() {
    return <h1>{this.state.message}, {this.props.name}</h1>;
  }
}

export default Welcome;
```

Example Program: A Class Component Counter

```
// Counter.jsx (Class Component)
import React from 'react';

class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 }; // initialize state
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
  }
}

export default Counter;
```

Output

Displays "Count: 0" initially; clicking "Increment" increases the count by 1 each time.

Real-World Example

Older versions of large applications, such as legacy banking dashboards built before 2019, often used class components to manage state and lifecycle behaviour before Hooks became the standard.

Advantages

- Supports state and lifecycle methods without needing Hooks.
- Well understood and widely documented due to years of prior use.
- Useful for understanding legacy React codebases.
- `this.props` and `this.state` provide clear, explicit access to data.

Limitations

- More verbose syntax compared to functional components with Hooks.
- The 'this' keyword binding can be confusing for beginners.
- Modern React development mostly favors functional components with Hooks.

Exam Focus

- Class components extend `React.Component`.
- The `render()` method is mandatory and must return JSX.
- State is initialized in the constructor and updated using `setState()`.
- Props are accessed via `this.props`; state via `this.state`.

Examination Questions

1-Mark Questions:

- 149. What is a class component?
- 150. Which method must every class component implement?
- 151. How is state initialized in a class component?
- 152. Which method is used to update state in a class component?

2-Mark Questions:

- 110. Differentiate between class components and functional components.
- 111. Explain how props and state are accessed in a class component.
- 112. Why is `this.setState()` used instead of modifying state directly?

4-Mark Questions:

- 76. Explain class components with syntax and a suitable example program.
- 77. Discuss the differences between class components and functional components with Hooks.

Module 19: React Lifecycle

Unit II: Introduction to React & JSX

Definition

The React component lifecycle refers to the series of phases a component goes through from creation to removal from the DOM: Mounting, Updating, and Unmounting.

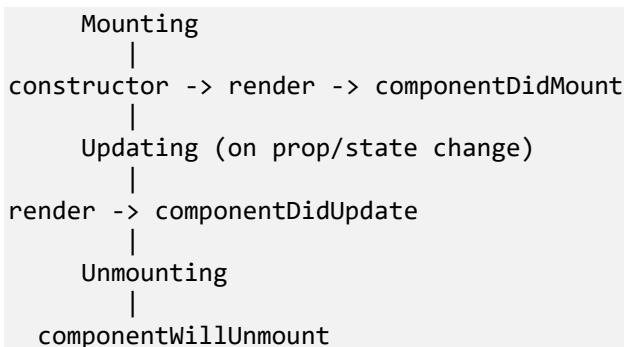
Why is it Needed?

- Helps developers run specific code at the right stage of a component's life, such as fetching data when it first appears.
- Enables cleanup of resources like timers or subscriptions when a component is removed.
- Essential for understanding how class components manage side effects.
- Forms the conceptual basis that `useEffect` in functional components is built upon.

Key Points

- Mounting phase: the component is created and inserted into the DOM (constructor, render, `componentDidMount`).
- Updating phase: the component re-renders due to changes in props or state (render, `componentDidUpdate`).
- Unmounting phase: the component is removed from the DOM (`componentWillUnmount`).
- `componentDidMount` runs once after the component is first rendered, often used for API calls.
- `componentDidUpdate` runs after every update, useful for responding to prop or state changes.
- `componentWillUnmount` runs just before the component is removed, used for cleanup.

Diagram / Flowchart



Syntax

```
class MyComponent extends React.Component {
  componentDidMount() {
    console.log("Component mounted");
  }

  componentDidUpdate() {
    console.log("Component updated");
  }

  componentWillUnmount() {
```

```

    console.log("Component will unmount");
  }

  render() {
    return <p>Lifecycle Demo</p>;
  }
}

```

Example Program: A Timer Component Using Lifecycle Methods

```

// Timer.jsx
import React from 'react';

class Timer extends React.Component {
  componentDidMount() {
    // Runs once when component mounts
    console.log("Timer started");
  }

  componentWillUnmount() {
    // Runs just before component is removed
    console.log("Timer stopped");
  }

  render() {
    return <p>Timer is running...</p>;
  }
}

export default Timer;

```

Output

Timer started	(logged when component mounts)
Timer stopped	(logged when component unmounts)

Real-World Example

A hospital management system's "Live Patient Monitor" component uses `componentDidMount` to start receiving real-time data when it appears on screen, and `componentWillUnmount` to stop the data feed when the nurse navigates away.

Advantages

- Provides precise control over what happens at each stage of a component's life.
- Enables proper cleanup, preventing memory leaks.
- Useful for tasks like API calls, subscriptions, and DOM measurements.
- Forms a strong conceptual foundation, even for developers using Hooks.

Limitations

- Lifecycle methods only exist in class components; functional components use `useEffect` instead.
- Understanding multiple, similarly-named methods can be confusing for beginners.

Exam Focus

- The three main phases are Mounting, Updating, and Unmounting.
- `componentDidMount` runs once after the initial render.

- `componentWillUnmount` runs just before a component is removed.
- `useEffect` in functional components serves a similar purpose to these lifecycle methods combined.

Examination Questions

1-Mark Questions:

153. Name the three phases of the React component lifecycle.
154. Which lifecycle method runs once after a component is first rendered?
155. Which lifecycle method runs just before a component is removed?
156. Which hook in functional components replaces class lifecycle methods?

2-Mark Questions:

113. Explain the Mounting phase of the React lifecycle.
114. Differentiate between `componentDidMount` and `componentDidUpdate`.
115. Why is `componentWillUnmount` important?

4-Mark Questions:

78. Explain the three phases of the React component lifecycle with a diagram and example.
79. Discuss how `useEffect` relates to traditional class lifecycle methods.

Module 20: React Context API

Unit II: Introduction to React & JSX

Definition

The Context API is a built-in React feature that allows data to be shared across multiple components without having to pass props manually at every level, solving the problem of "prop drilling."

Why is it Needed?

- Avoids passing props through many intermediate components that don't need the data themselves, known as prop drilling.
- Simplifies sharing of global data, such as themes, authentication status, or language settings.
- Makes code cleaner and easier to maintain in deeply nested component trees.
- Provides a built-in alternative to external state management libraries for simpler use cases.

Key Points

- createContext() creates a new Context object.
- The Provider component supplies the context value to all its descendant components.
- The Consumer component, or the useContext hook, is used to access the context value.
- useContext is the modern, simpler way to consume context in functional components.
- Context should be used for data that is genuinely global, such as theme or user info, not for all state.

Diagram / Flowchart

```

    <ThemeContext.Provider value="dark">
      |
    App Component
    /      \
Header      Sidebar
|           |
useContext()  useContext()
(gets "dark" without props being passed manually)
```

Syntax

```
import { createContext, useContext } from 'react';

// Create context
const ThemeContext = createContext();

// Provide context
<ThemeContext.Provider value="dark">
  <App />
</ThemeContext.Provider>

// Consume context using useContext
const theme = useContext(ThemeContext);
```

Example Program: Sharing a Theme Value Across Components

```
// ThemeContext.js
import { createContext } from 'react';
```

```
export const ThemeContext = createContext("light");

// App.jsx
import { ThemeContext } from './ThemeContext';
import Header from './Header';

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Header />
    </ThemeContext.Provider>
  );
}

// Header.jsx
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

function Header() {
  const theme = useContext(ThemeContext); // access context directly
  return <h2>Current Theme: {theme}</h2>;
}

export default Header;
```

Output

Current Theme: dark

Real-World Example

A learning management system uses the Context API to share the logged-in user's information, such as name and role, across many components, such as the Navbar, Dashboard, and Profile page, without passing it manually through every component.

Advantages

- Eliminates the need for prop drilling through many nested components.
- Simplifies sharing of global data across an application.
- Built into React, requiring no additional library for basic use cases.
- Works well with the useContext Hook for clean, readable code.

Limitations

- Not ideal for very frequently changing, large-scale state, where external libraries may perform better.
- Overusing Context for all state can make components harder to test in isolation.

Exam Focus

- The Context API solves the problem of prop drilling.
- createContext(), Provider, and useContext are the key building blocks.
- useContext is the modern way to consume context in functional components.
- Context should be used only for genuinely global or shared data.

Examination Questions

1-Mark Questions:

- 157. What problem does the Context API solve?
- 158. Which function is used to create a context?
- 159. Which component supplies the context value to its children?
- 160. Which hook is used to consume context in functional components?

2-Mark Questions:

- 116. Explain the purpose of the Provider component in the Context API.
- 117. What is prop drilling, and how does the Context API help avoid it?
- 118. Give an example of data suitable for sharing via Context.

4-Mark Questions:

- 80. Explain the React Context API with syntax, diagram, and a suitable example.
- 81. Discuss the advantages and limitations of using the Context API for state sharing.

Module 21: Lazy Loading

Unit II: Introduction to React & JSX

Definition

Lazy loading in React is a technique that delays loading certain components until they are actually needed, using `React.lazy()` combined with code splitting to improve initial application performance.

Why is it Needed?

- Reduces the initial bundle size, leading to faster first page loads.
- Loads components only when required, saving bandwidth and load time.
- Improves performance for large applications with many components or pages.
- Enhances user experience, especially on slower networks or devices.

Key Points

- `React.lazy()` is used to dynamically import a component.
- Works together with `Suspense` to show a fallback UI while the component loads.
- Primarily used for code splitting at the route or component level.
- Only supports default exports when using `React.lazy()`.
- Reduces the size of the initial JavaScript bundle sent to the browser.

Diagram / Flowchart

```
graph TD
    A[Initial Bundle (small, fast to load)] --> B[User navigates to a route]
    B --> C[React.lazy() dynamically imports the component]
    C --> D[Component code downloaded and rendered]
```

Syntax

```
import React, { lazy } from 'react';

// Lazily import a component
const Dashboard = lazy(() => import('./Dashboard'));
```

Example Program: Lazily Loading a Dashboard Component

```
// App.jsx
import React, { lazy, Suspense } from 'react';

// Lazily load the Dashboard component
const Dashboard = lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback=<p>Loading Dashboard...</p>>
      <Dashboard />
    </Suspense>
  );
}
```

```
export default App;
```

Output

Displays "Loading Dashboard..." briefly, then shows the Dashboard component once it finishes loading.

Real-World Example

An online food delivery app lazily loads the "Order History" page only when the user navigates to it, keeping the initial app load, such as the Home page, fast and lightweight.

Advantages

- Reduces initial load time by splitting code into smaller chunks.
- Improves performance for large-scale applications.
- Loads only the code that is actually needed at a given time.
- Works seamlessly with Suspense for a smooth loading experience.

Limitations

- Requires careful planning of which components to lazy load.
- Can introduce brief loading delays when a lazily loaded component is first accessed.

Exam Focus

- `React.lazy()` is used for dynamically importing components.
- Lazy loading must be used together with Suspense.
- Only default exports are supported by `React.lazy()`.
- Reduces the initial JavaScript bundle size.

Examination Questions

1-Mark Questions:

161. What is lazy loading in React?
162. Which function is used to lazily import a component?
163. Which component must be used alongside `React.lazy()`?
164. What type of export does `React.lazy()` support?

2-Mark Questions:

119. Explain the purpose of code splitting in React.
120. How does lazy loading improve application performance?
121. Give a real-world example of when lazy loading would be useful.

4-Mark Questions:

82. Explain lazy loading in React with syntax and a suitable example.
83. Discuss the advantages and limitations of using `React.lazy()` in large applications.

Module 22: Suspense

Unit II: Introduction to React & JSX

Definition

Suspense is a built-in React component that lets you display a fallback UI, such as a loading indicator, while waiting for lazily loaded components, or other asynchronous operations, to finish loading.

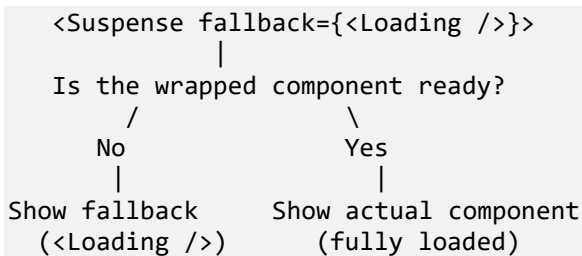
Why is it Needed?

- Provides a clean way to show loading states without manually managing loading flags.
- Works together with `React.lazy()` to handle code-split components gracefully.
- Improves perceived performance and user experience during loading.
- Simplifies handling of asynchronous UI states in a declarative way.

Key Points

- Suspense wraps components that might not be ready immediately, such as lazily loaded components.
- The `fallback` prop specifies what to display while waiting, such as a spinner or a message.
- Commonly used together with `React.lazy()` for code-split components.
- Can wrap multiple lazy components under a single fallback.
- Newer React versions are expanding Suspense to support data fetching as well.

Diagram / Flowchart



Syntax

```
import { Suspense } from 'react';

<Suspense fallback={<p>Loading...</p>}>
  <LazyComponent />
</Suspense>
```

Example Program: Showing a Loading Message While a Profile Loads

```
// App.jsx
import React, { lazy, Suspense } from 'react';

const Profile = lazy(() => import('./Profile'));

function App() {
  return (
    <div>
      <h1>My App</h1>
      <Suspense fallback={<p>Loading Profile...</p>}>
        <Profile />
      </Suspense>
    </div>
  );
}
```

```
    </Suspense>
  </div>
);
}

export default App;
```

Output

Displays "My App" immediately, along with "Loading Profile..." briefly, before the Profile component appears once it has fully loaded.

Real-World Example

A social media application shows a loading spinner using Suspense while the "Stories" component, loaded lazily, fetches and prepares its content, giving users clear feedback instead of a blank screen.

Advantages

- Provides a clean, declarative way to handle loading states.
- Improves user experience by avoiding blank screens during loading.
- Works seamlessly with React.lazy() for code-split components.
- Can wrap multiple components under a single fallback UI.

Limitations

- Primarily designed for code-split components; broader data-fetching support is still evolving.
- Improper use, such as a missing fallback, can lead to unhandled loading states.

Exam Focus

- Suspense displays a fallback UI while components are loading.
- Commonly used together with React.lazy().
- The fallback prop defines what is shown during loading.
- Suspense simplifies handling of asynchronous UI states.

Examination Questions

1-Mark Questions:

165. What is the purpose of the Suspense component?
166. Which prop is used to specify the loading UI in Suspense?
167. Which React feature is Suspense commonly used together with?
168. Can Suspense wrap multiple lazy components?

2-Mark Questions:

122. Explain how Suspense works with React.lazy().
123. What happens if the fallback prop is not provided in Suspense?
124. Give an example of a suitable fallback UI for Suspense.

4-Mark Questions:

84. Explain the Suspense component with syntax and a suitable example program.
85. Discuss how Lazy Loading and Suspense work together to improve application performance.